

CASE STUDY ARCHITETTURALE

Hetepi AI

Costruire un agente AI personale con OpenClaw

Architettura, controllo dei costi e replicabilità - dallo zero allo stato attuale

Un resoconto del processo di costruzione di un ecosistema AI personale: architettura, routing dei modelli, controllo dei costi, automazioni, memoria persistente e il modulo Portfolio Intelligence come caso d'uso avanzato.

AUTORE

Andrea Sforna

PIATTAFORMA

macOS - Intel Core i7

DATA

19 giugno 2026

ASSISTENTE

Hetepi

VERSIONE

1.1 (revisione estesa)

RISERVATEZZA

Personale

Indice

Indice	2
1. Executive Summary	3
2. Obiettivo iniziale	5
3. Installazione e setup iniziale	6
4. Architettura dell'agente	8
5. Modelli AI utilizzati e routing	9
6. Routing dei modelli e modalità operative	10
7. Strategia costi e token	13
8. Tecniche di riduzione costi e token	17
9. Cosa è deterministico e cosa è AI	19
10. Memoria e personalizzazione	20
11. Integrazioni e sistemi controllati	21
12. Portfolio Intelligence come caso studio avanzato	22
13. Problemi affrontati e soluzioni	23
14. Metodo di sviluppo	24
15. Risultati raggiunti	25
16. Architettura attuale in una pagina	26
17. Stato attuale e prossimi passi	27
18. Replicabilità in azienda	28
19. Evoluzione delle decisioni tecniche	31
20. Principi architetturali e scelte progettuali	33
Appendice	35

Convenzione di lettura - In tutto il documento le affermazioni sono marcate per livello di certezza: **[verificato]** (confermato da codice, configurazione, git o memoria dell'agente), **[ricostruzione]** (dedotto da fonti indirette, non direttamente ispezionabile) e **[stima]** (ordine di grandezza non misurato). Dove un dato non è verificabile, è dichiarato come tale invece di essere inventato.

1. Executive Summary

Questo documento racconta la costruzione di **Hetepi AI**: un agente di intelligenza artificiale personale, operativo e residente su un Mac, realizzato sopra la piattaforma **OpenClaw** e governato interamente da Telegram. Non è un chatbot che si limita a rispondere: è un sistema che ricorda, agisce, analizza e automatizza, con un'attenzione costante a due vincoli che hanno guidato ogni decisione: il controllo dei costi e la privacy.

L'agente è raggiungibile da un'unica interfaccia conversazionale (un bot Telegram privato) e dietro quella interfaccia coordina: più modelli AI cloud e locali con un sistema di routing esplicito; una memoria persistente su file che sopravvive ai riavvii; un insieme di automazioni deterministiche schedulate con il gestore di servizi nativo di macOS (launchd); il controllo di dispositivi di casa (TV, robot aspirapolvere, luci); l'accesso a Google Workspace; sintesi e trascrizione vocale; e un modulo verticale complesso, Portfolio Intelligence, che da solo dimostra fino a che punto l'agente abbia abilitato la costruzione di software reale.

Cosa è stato costruito, in sintesi

- Un **agente operativo su Telegram**: comandi in linguaggio naturale che si traducono in azioni reali sul Mac e sui servizi collegati. [verificato]
- Un **sistema di routing dei modelli** con modalità a costo zero (modelli locali via Ollama), modello cloud economico di default (DeepSeek) e modelli premium (Claude) solo su richiesta esplicita. [verificato]
- Una **memoria a due livelli**: note giornaliere grezze e memorie curate a lungo termine, che codificano preferenze, regole operative e storia del progetto. [verificato]
- Una **rete di automazioni deterministiche** (16 servizi launchd) per report, backup cifrati, pulizia, monitoraggio e tracciamento spesa, eseguite a costo AI nullo. [verificato]
- Il modulo **Portfolio Intelligence**: un bot multi-utente che fa analisi di portafoglio, scoring, stima del rischio, simulazioni Monte Carlo e reportistica, costruito quasi interamente in Python deterministico. [verificato]

Risultati principali

Dimensione	Prima	Dopo
Natura del sistema	Chatbot che risponde a domande	Agente che esegue azioni reali e automazioni
Memoria	Nessuna continuità tra sessioni	Memoria persistente su file, curata e indicizzata
Costi AI	Potenzialmente migliaia di \$/mese a modello premium	Routing su locale/economico: ordine di pochi euro/giorno [stima]
Analisi finanziaria	Calcoli manuali, foglio per foglio	Bot automatico multi-utente, report schedulati
Operatività	Azioni manuali una a una	Automazioni schedulate, backup cifrati, alert

Tabella 1 - Trasformazione sintetica dell'ecosistema (dettaglio nel cap. 15).

Schema generale dell'ecosistema

Diagramma 1 - Vista a volo d'uccello

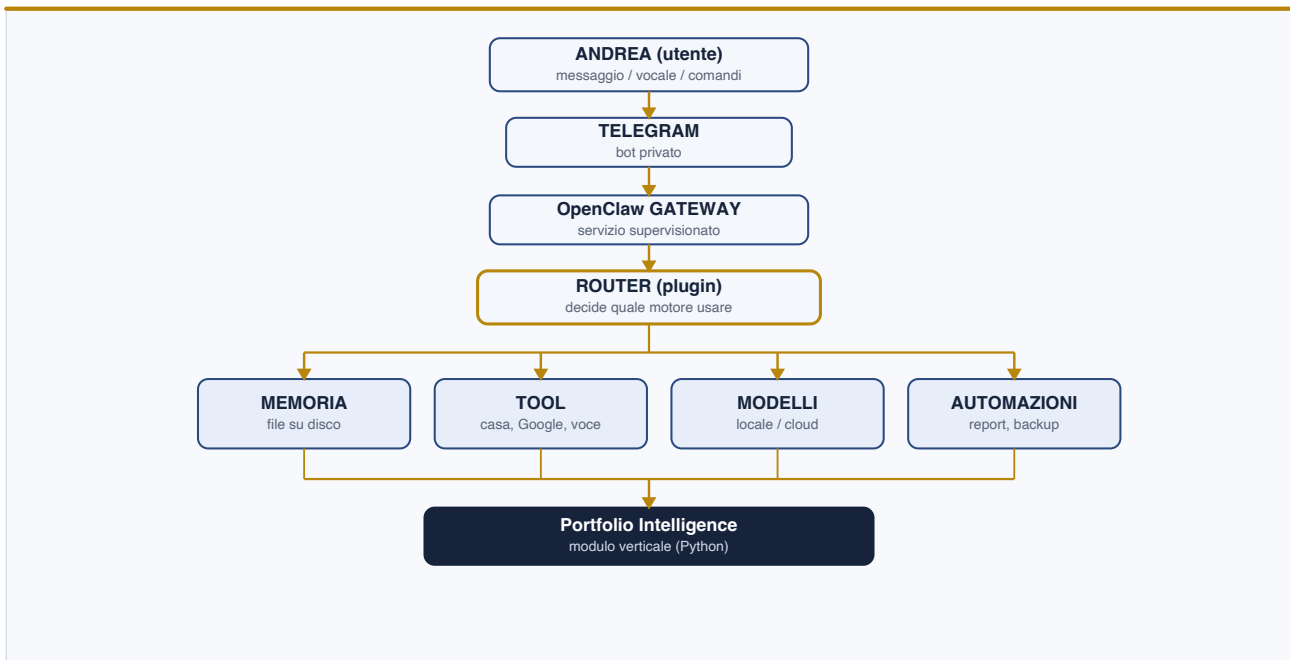


Diagramma 1 - Ingresso unico (Telegram), un router che decide quale motore usare, e quattro pilastri (memoria, tool, modelli, automazioni).

2. Obiettivo iniziale

Il progetto nasce da un'esigenza precisa: non volevo un chatbot, volevo un **agente**. La differenza è sostanziale. Un chatbot produce testo; un agente percepisce un contesto, decide, e compie azioni nel mondo reale - invia messaggi, legge la posta, accende una luce, genera un report, fa un backup - e poi rende conto di ciò che ha fatto.

Perché è nato il progetto

L'obiettivo era avere un assistente personale operativo, controllabile da un'unica interfaccia comoda e sempre a portata di mano - Telegram - capace di:

- **Ricordare**: mantenere il contesto su di me, le mie preferenze e la storia del lavoro, senza ripartire da zero a ogni conversazione.
- **Agire**: eseguire comandi concreti sul Mac e sui dispositivi collegati, non solo descrivere come si farebbe.
- **Rispondere**: in linguaggio naturale, in italiano, con uno stile asciutto e senza riempitivi.
- **Analizzare**: elaborare dati reali (in primis finanziari) e restituire sintesi affidabili.
- **Automatizzare**: svolgere da solo, su pianificazione, i compiti ricorrenti.

I vincoli di progetto

Tre vincoli, fissati all'inizio, hanno agito da bussola in ogni scelta tecnica successiva. Non erano dettagli implementativi: erano principi architetturali.

Vincolo	Significato	Conseguenza architettuale
Costo	La spesa per i modelli AI non deve sfuggire di mano	Routing esplicito dei modelli, default economico, locale dove possibile, lavoro deterministico fuori dall'AI
Privacy	I dati personali restano sul mio Mac e non vengono esfiltrati	Memoria su file locali, media in ingresso cancellati dopo l'uso, modelli locali per ciò che è sensibile
Locale / controllo	Voglio possedere e capire il sistema, non dipendere da una scatola nera	Esecuzione sul Mac con launchd, configurazione ispezionabile, componenti sostituibili

Tabella 2 - I tre vincoli fondativi e il loro impatto sul design.

Questi vincoli spiegano molte decisioni che altrimenti sembrerebbero arbitrarie: perché esista una modalità a costo zero, perché i calcoli finanziari non passino dall'AI, perché i media ricevuti vengano cancellati automaticamente, e perché ogni passaggio a un modello più costoso richieda una conferma esplicita.

3. Installazione e setup iniziale

Questa sezione ricostruisce il setup tecnico del sistema. Lo stato attuale dei file, dei servizi e della configurazione è verificato per ispezione diretta; la sequenza esatta dei comandi di prima installazione non è interamente registrata, quindi i comandi di bootstrap sono indicati come **[ricostruzione]** - plausibili e coerenti con lo stato finale, ma non garantiti alla lettera.

Ambiente di base

- **Hardware/OS:** Mac con macOS, su piattaforma Intel Core i7, senza GPU dedicata per l'inferenza. [verificato]
- **Runtime:** Node.js (per OpenClaw) e Python 3 di sistema (per le automazioni e il modulo Portfolio). [verificato]
- **Inferenza locale:** Ollama installato come servizio locale. [verificato]
- **Interfaccia:** un bot Telegram privato come unico canale di interazione. [verificato]

Installazione di OpenClaw

OpenClaw è distribuito come pacchetto Node ed è installato globalmente; il gateway gira come servizio launchd permanente. La tabella seguente riporta operazioni e scopo nel formato comando/operazione -> scopo -> risultato.

Operazione	Scopo	Risultato / stato
npm i -g openclaw [ricostruzione]	Installare la piattaforma agente a livello di sistema	Binario installato a livello di sistema [verificato]
openclaw gateway via launchd	Avviare il gateway come servizio sempre attivo	Servizio gateway attivo e con KeepAlive [verificato]
Creazione workspace	Cartella di lavoro dell'agente con file di identità e memoria	~/openclaw/workspace popolata (AGENTS.md, SOUL.md, memory/) [verificato]
Pairing canale Telegram	Collegare il bot privato e l'utente autorizzato	Solo il chat_id autorizzato dialoga con l'agente [verificato]
Installazione Ollama + pull modelli	Abilitare inferenza locale a costo zero	qwen3:8b, varianti abilitate e deepseek-ocr scaricati [verificato]

Tabella 3 - Passi di installazione (i comandi marcati [ricostruzione] sono plausibili, non registrati).

Struttura delle cartelle

Diagramma 2 - Albero delle cartelle principali

~/openclaw/	
└─ openclaw.json	configurazione principale (modelli, canali, plugin)
└─ service-env/	wrapper e file env per i servizi supervisionati
└─ agents/main/	
└─ sessions/*.jsonl	transcript per-sessione (token e costi)
└─ agent/plugins/<prov>/	catalog.json: chiavi API + modelli
└─ workspace/	la 'casa' dell'agente
└─ AGENTS.md SOUL.md	operatività e persona
└─ USER.md IDENTITY.md	profilo utente e identità
└─ MEMORY.md	regole core sempre caricate
└─ memory/	note giornaliera + changelog
└─ device-notes/	schede dispositivi (TV, robot, voce)
└─ plugins/gratis-local/	routing verso i modelli locali
└─ portfolio_*.py	modulo Portfolio Intelligence

Diagramma 2 - I nomi sono verificati; il contenuto sensibile non è mostrato.

Configurazione iniziale e segreti

La configurazione vive in **openclaw.json** (modelli ammessi, canali, plugin) e nei catalog per-provider dei plugin. Un dettaglio appreso sul campo è rilevante per chi replicasse il setup: le chiavi API dei provider non stanno in openclaw.json, ma nei file agents/main/agent/plugins/<provider>/catalog.json. Dedurre lo stato di un provider dal solo openclaw.json induce in errore (è successo davvero, con due falsi allarmi chiave mancante poi smentiti). [verificato]

I segreti - token del bot, chiavi dei provider, chiave per la web search di Ollama, passphrase di backup - risiedono in file a permessi ristretti (chmod 600) o nella configurazione. In questo documento nessun valore segreto è riportato: si descrive solo che esistono e a cosa servono.

Nota metodologica. Lo stato finale del sistema è ispezionabile e quindi verificato. La cronologia git del workspace parte dai primi di giugno 2026 e riguarda soprattutto il modulo Portfolio; la data esatta della prima installazione di OpenClaw non è registrata e va quindi considerata una ricostruzione.

HETEPI AI · ECOSISTEMA OPENCLAW

4. Architettura dell'agente

L'architettura segue un principio semplice: un solo ingresso, un router che decide, un insieme di capacità (modelli, tool, memoria, automazioni) che il router può attivare. Tutto ciò che l'utente vede è una conversazione Telegram; tutto il resto è orchestrazione interna.

Il percorso di una richiesta

- **1. Arrivo.** Un messaggio (testo o vocale) entra dal bot Telegram. Solo il chat_id autorizzato è servito; i gruppi sono esclusi per le azioni sensibili. [verificato]
- **2. Ricezione.** Il gateway OpenClaw riceve l'evento e lo passa alla catena di dispatch.
- **3. Instradamento.** Un hook before_dispatch (il plugin di routing) ispeziona il messaggio: se è in modalità locale o ha un prefisso dedicato, risponde col modello locale e si ferma; altrimenti lascia proseguire verso il modello cloud di default.
- **4. Modello / tool.** Il modello scelto ragiona e, se serve, invoca i tool (casa smart, Google, voce, file, ecc.) per compiere azioni reali.
- **5. Risposta.** L'output torna su Telegram nel formato richiesto (testo o, se l'input era un vocale, un vocale).
- **6. Memorizzazione.** Ciò che è utile al futuro viene scritto su file di memoria; i media in ingresso vengono cancellati dopo l'uso.

Diagramma architetturale

Diagramma 3 - Flusso richiesta -> AI/tool -> azione -> risposta

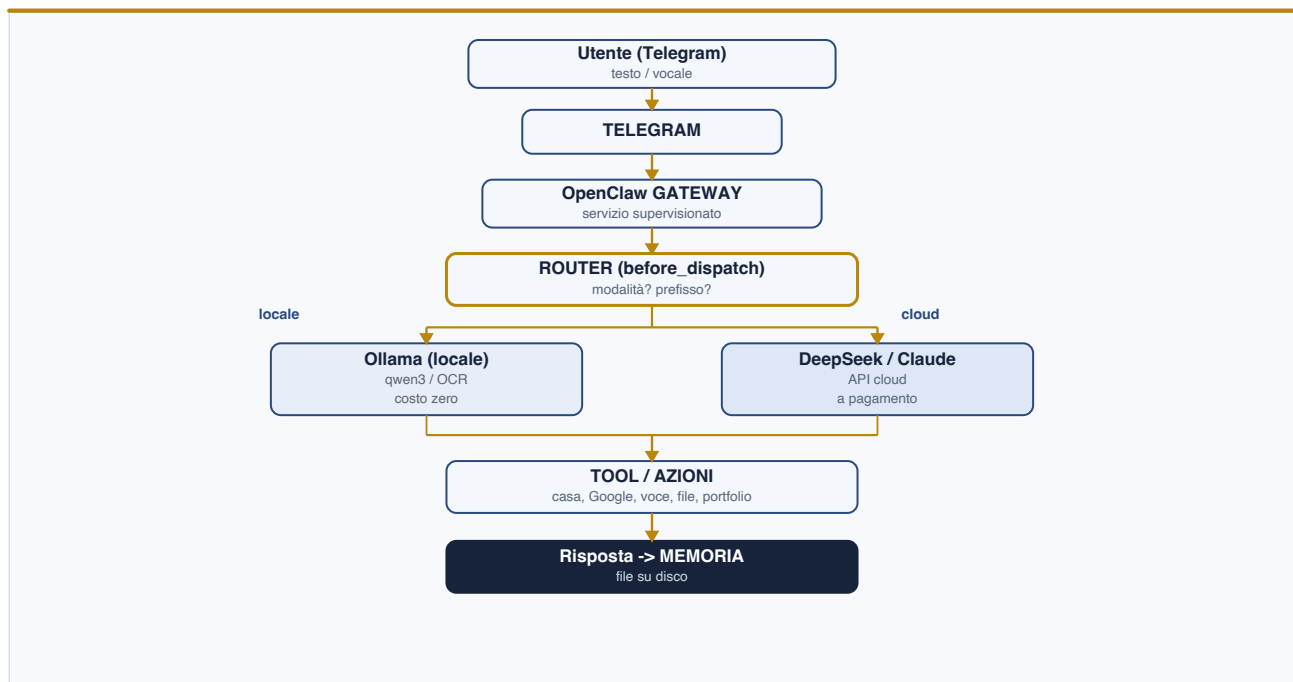


Diagramma 3 - Il router separa il percorso locale da quello cloud.

Cosa viene memorizzato e cosa no

Vengono memorizzate le informazioni con valore di continuità: preferenze, regole operative, decisioni di progetto, stato di lavori in corso. Non vengono conservati i media in ingresso (immagini, vocali): una volta analizzati e risposti, sono cancellati da un'automazione dedicata. È una scelta di privacy esplicita, non un effetto collaterale. [verificato]

5. Modelli AI utilizzati e routing

Il sistema non usa un modello: usa una **famiglia di modelli**, ciascuno con un ruolo definito, scelto in base al rapporto tra costo, qualità e località (cloud vs locale). La regola generale è: il modello entra per compito, non per turno - non si fa ping-pong di provider a ogni messaggio, perché cambiare provider a ogni turno romperebbe la cache e farebbe lievitare i costi.

I modelli in gioco

- **DeepSeek V4 Flash (cloud)** - motore di default: chat, comandi di casa, domande leggere. Il più economico. [verificato]
- **DeepSeek V4 Pro (cloud)** - compiti pesanti: programmazione, fix dei bot, analisi. Si attiva solo dopo conferma esplicita, mai in automatico. [verificato]
- **Claude (Opus 4.8 / Fable 5, cloud premium)** - fuori dalla scala automatica: si usa solo se richiesto a mano via comando. Qualità massima, costo elevato. [verificato]
- **Ollama locale (qwen3:8b e variante abilitated)** - modalità a costo zero, gira interamente sul Mac. [verificato]
- **DeepSeek-OCR (locale, via Ollama)** - visione su immagini/PDF/scansioni; necessario perché l'API DeepSeek è solo testo. [verificato]
- **Whisper (locale)** - trascrizione vocale (speech-to-text) senza chiavi API. [verificato]
- **Piper / ElevenLabs** - sintesi vocale (text-to-speech): Piper è locale e gratuito, ElevenLabs cloud entro la quota gratuita. [verificato]

Tabella dei modelli

Modello	Uso	Locale/Cloud	Costo (in/out \$/Mtok)	Vantaggio	Limite
DeepSeek V4 Flash	Default, routine	Cloud	0,14 / 0,28	Costo bassissimo, buono sul quotidiano	Meno forte su compiti complessi
DeepSeek V4 Pro	Codice, analisi	Cloud	0,435 / 0,87	Qualità alta a costo contenuto	Solo on-demand, va confermato
Claude (Opus/Fable)	Compiti critici	Cloud	5 / 25 (Opus)	Qualità di punta	Costo elevato, mai default
qwen3:8b (Ollama)	Modalità gratis	Locale	0 (gratis)	Zero costo, dati in casa	Più lento su CPU, meno preciso
DeepSeek-OCR	Visione/OCR	Locale	0 (gratis)	Legge testo da immagini/PDF	Debole sulle foto da capire
Whisper	Voce -> testo	Locale	0 (gratis)	Trascrizione senza API	Carico CPU
Piper / ElevenLabs	Testo -> voce	Locale / Cloud	0 / quota free	Voce naturale, fallback locale	ElevenLabs ha quota mensile

Tabella 4 - Mappa dei modelli. Prezzi verificati a giugno 2026 sulle pagine ufficiali dei provider.

La scelta del default DeepSeek deriva da un'analisi concreta: misurando i token reali consumati, il modello premium in uso continuo sarebbe costato un ordine di grandezza di migliaia di dollari al mese (driver dominante: la rilettura del contesto a ogni turno). Spostare la routine su un modello economico cloud, e il banale sul locale, riporta la spesa a pochi euro al giorno. La sezione 6 descrive in dettaglio il meccanismo di routing.

6. Routing dei modelli e modalità operative

Questa è una delle parti più caratterizzanti del sistema. Non si tratta soltanto di usare Claude, DeepSeek e Ollama: è un meccanismo esplicito di scelta del motore, pensato per risparmiare, avere sempre una via a costo zero, e mantenere la qualità alta dove conta. Il cuore del routing è un plugin OpenClaw che intercetta i messaggi prima che raggiungano qualsiasi modello a pagamento.

Modalità disponibili

- **Modalità normale** - nessun comando speciale: il messaggio va al modello cloud di default (DeepSeek V4 Flash). È lo stato base.
- **Prefisso one-shot gratis:<domanda>** - una singola domanda risposta dal modello locale, senza memoria, a costo zero. Non cambia lo stato della chat.
- **gratis on** - modalità persistente: da quel momento tutti i messaggi vanno al modello locale standard (qwen3:8b), con memoria conversazionale, finché non si esce.
- **gratis libero** - come sopra, ma usa il modello locale abliterated (senza filtri). Entra direttamente in modalità con un solo comando.
- **gratis normale** - a sessione aperta, torna al modello locale standard restando in modalità.
- **gratis off** - esce dalla modalità locale, cancella la memoria della chat e riporta tutto all'agente normale. Funziona sempre, anche se già spento (così non si resta mai bloccati).
- **Premium on-demand** - Claude/DeepSeek-Pro non hanno un comando di modalità: si attivano per compito, con conferma esplicita.

Cosa succede quando attivo una modalità

Modalità / comando	Modello	Loc./Cloud	Costo	Velocità	Qualità	Uso consigliato
normale (default)	DeepSeek Flash	Cloud	Molto basso	Veloce	Buona	Quotidiano, comandi, domande
gratis: (one-shot)	qwen3:8b	Locale	Zero	Pochi secondi	Media	Domanda secca a costo zero
gratis on	qwen3:8b	Locale	Zero	Pochi s; ~1-2 min con ricerca	Media	Sessione privata a costo zero
gratis libero	qwen3-abliterated 8b	Locale	Zero	Simile allo standard	Media, senza filtri	Quando servono risposte non filtrate
on-demand pro	DeepSeek Pro	Cloud	Medio	Veloce	Alta	Codice, analisi (con conferma)
on-demand premium	Claude	Cloud	Alto	Veloce	Massima	Compiti critici (a richiesta)

Tabella 5 - Modalità -> modello -> località -> costo -> velocità -> qualità -> uso. Tempi locali misurati sul Mac (CPU Intel, no GPU); i tempi cloud non sono stati cronometrati [stima].

Lo script di routing

La logica vive nel plugin **plugins/gratis-local/index.js**, agganciato all'evento `before_dispatch` con priorità alta. È stato scelto questo hook (e non `inbound_claim`) perché `inbound_claim` scatta solo per i plugin che possiedono un binding di conversazione, e quindi non si attivava mai sui messaggi diretti; `before_dispatch` scatta invece su ogni messaggio in ingresso. [verificato]

Cosa legge e cosa modifica

- **Legge** il testo del messaggio, il mittente (deve essere il `chat_id` autorizzato) e se è un gruppo (in tal caso non interviene).
- **Legge lo stato** da tre file: `.gratis_mode` (on/off), `.gratis_model` (standard/libero) e `.gratis_history.json` (memoria di chat, max 6 scambi).
- **Decide**: se riconosce un comando di modalità lo applica e risponde; se è in modalità (o c'è il prefisso) gestisce la domanda in locale; altrimenti restituisce il controllo all'agente cloud.
- **Modifica** i file di stato quando si entra/esce da una modalità, e azzera la memoria a ogni accensione e spegnimento (scelta di privacy).

Come evita le chiamate costose e come torna indietro

Quando il plugin gestisce un messaggio restituisce `{ handled: true, text }`: a quel punto il core consegna la risposta e ritorna senza invocare alcun modello a pagamento. È così che la modalità `gratis` garantisce davvero costo zero. Per tornare alla modalità precedente basta `gratis off`, che ripristina l'agente normale, cancella la memoria locale e resetta il modello a standard. [verificato]

Web search dal locale

Anche in modalità locale l'agente può accedere a dati aggiornati: il modello locale dispone di un tool `web_search` e, quando lo ritiene utile, il plugin interroga l'API di web search di Ollama (cloud, gratuita con chiave) e ripassa i risultati al modello, che risponde basandosi solo su quelli. Massimo 3 round di tool, timeout complessivo 180 s. [verificato]

Fallback

La resilienza è organizzata su livelli, con un principio fermo: il fallback automatico contiene solo modelli economici o gratuiti. DeepSeek-Pro e Claude non entrano mai nel fallback automatico - un upgrade verso un modello più caro è sempre una decisione esplicita.

Livello	Modello	Quando interviene
Principale	DeepSeek V4 Flash (cloud)	Stato normale, default di ogni richiesta
Fallback economico	Modello cloud economico alternativo	Se il principale non risponde (resilienza)
Fallback locale	qwen3:8b via Ollama	Se il cloud è irraggiungibile: si resta operativi a costo zero
Upgrade (manuale)	DeepSeek Pro -> poi Claude	Solo su richiesta esplicita per compiti difficili; mai automatico

Tabella 6 - Catena di fallback. La salita verso modelli costosi non è mai automatica.

Cosa succede in caso di errore: se il modello locale non risponde (Ollama spento o in errore), il plugin restituisce un messaggio chiaro che invita a riprovare o a passare a DeepSeek/Claude, invece di fallire in silenzio. Se l'API cloud fallisce, intervengono i fallback economici. [verificato]

Diagramma del routing

Diagramma 4 - Router: a ogni richiesta il modello adeguato (costo, qualità, velocità, locale/cloud)

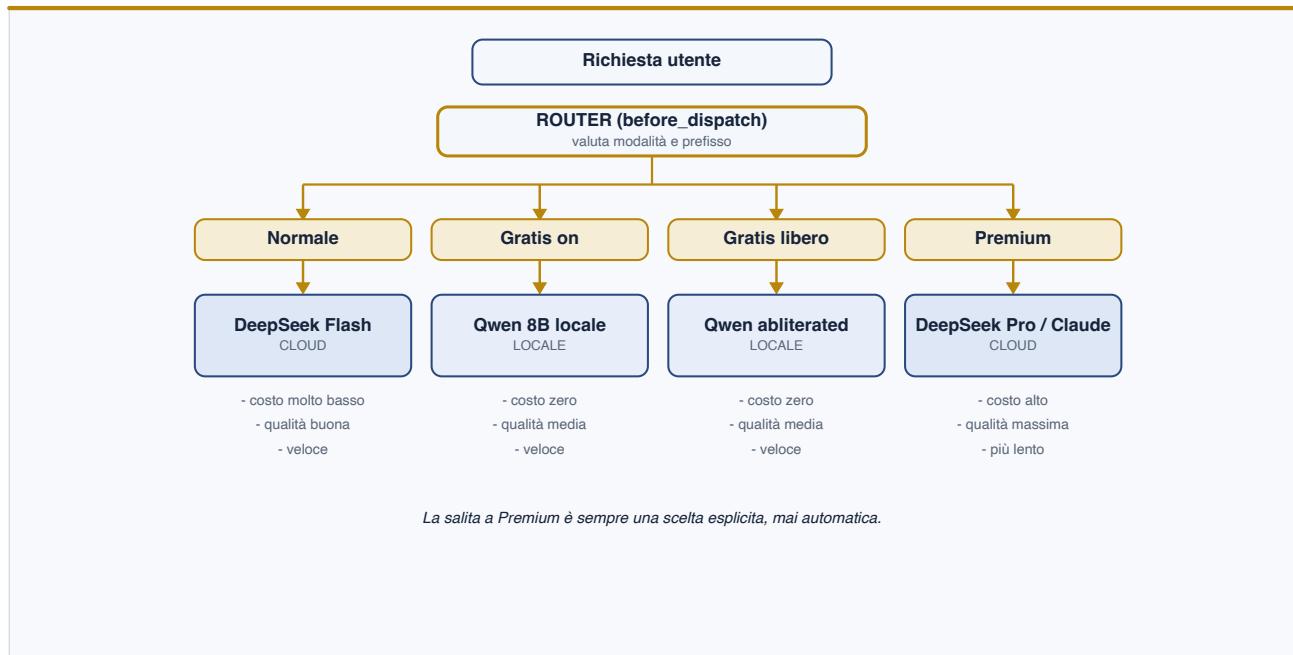


Diagramma 4 - Il percorso completo di instradamento, dal comando alla risposta.

Perché l'ho impostato così

La filosofia del routing si riassume in cinque punti: (1) contenere i costi come vincolo, non come ripensamento; (2) usare il locale quando è sostenibile, così da avere sempre una via gratuita e privata; (3) usare il cloud economico per la routine; (4) riservare i modelli premium ai compiti dove la qualità fa davvero la differenza, e solo su decisione esplicita; (5) non sprecare token - niente chiamate AI per ciò che è già deterministico.

Esempi concreti

- Se scrivo **gratis on**, il sistema passa al modello locale qwen3:8b: da lì ogni messaggio è gestito in casa, a costo zero, finché non esco. [verificato]
- Se uso **gratis libero**, il sistema entra in modalità locale col modello abilitated, senza filtri. [verificato]
- Se un compito è troppo difficile per il default, l'agente lo segnala e propone di salire a DeepSeek-Pro; sale solo dopo il mio ok. [verificato]
- Se la richiesta è semplice, resta sul default economico senza scomodare nulla di più caro. [verificato]

Nota metodologica. Le regole di attivazione qui descritte corrispondono al codice del plugin alla data del documento. Se il plugin viene modificato in futuro, questa sezione va riverificata: la fonte di verità è sempre il file `index.js` del plugin.

Collegamento con il resto dell'ecosistema

Il routing non è un dettaglio isolato: condiziona l'intero sistema. Su Telegram determina chi risponde a ogni messaggio; in OpenClaw agisce prima del dispatch ai modelli; per le automazioni impone di usare modelli economici (o nessun modello) sui cron; per la voce abilita risposte vocali anche in locale; per Portfolio Intelligence garantisce che l'analisi pesante resti deterministica e non bruci token; è nel complesso il principale meccanismo di contenimento dei costi e di stabilità del sistema. In altre parole, non ho solo usato dei modelli: ho costruito un sistema di scelta, risparmio, fallback e controllo operativo.

7. Strategia costi e token

Il controllo dei costi è un vincolo architetturale, non un'ottimizzazione tardiva. La leva principale è stata capire dove finiscono davvero i soldi: nei modelli premium, il costo dominante non è la singola risposta ma la rilettura del contesto a ogni turno (cache read). Da qui tutte le contromisure.

Task -> modello -> perché -> costo -> alternativa evitata

Tipo di task	Modello scelto	Perché	Costo stimato	Alternativa più cara evitata
Chat, comandi casa	DeepSeek Flash	Sufficiente ed economico	Bassissimo	Claude Opus (~x18 in input)
Domanda banale/privata	qwen3:8b locale	Costo zero, dati in casa	Zero	Qualunque API cloud
Codice / fix bot	DeepSeek Pro (su ok)	Qualità alta, costo contenuto	Medio	Claude Opus
Compito critico	Claude (a richiesta)	Qualità di punta dove serve	Alto	-
Calcoli finanziari	Nessun modello (Python)	Deterministico, ripetibile	Zero	Qualunque modello AI
Report schedulati	Nessun modello (Python)	Output fisso, niente AI	Zero	Cron con modello AI

Tabella 8 - Per ogni tipo di task, il modello scelto è l'alternativa più costosa evitata.

Problema, soluzione, beneficio

Letta come sequenza di decisioni, la strategia costi è una catena di problemi concreti e di contromisure mirate. Il punto di svolta è stato il **cambio di billing di Anthropic**: con la scadenza del piano Max, mantenere Claude come modello sempre attivo sarebbe costato di continuo a tariffa piena. Da lì la riprogettazione che ha reso DeepSeek il modello operativo principale e Claude una risorsa su richiesta. La tabella riassume ogni problema e cosa ha prodotto la soluzione adottata.

Problema	Soluzione adottata	Beneficio ottenuto
Cambio billing Anthropic: Claude default insostenibile	DeepSeek come modello operativo, Claude solo on-request	Spesa di base crollata, qualità premium dove serve
Il modello premium rilegge tutto a ogni turno	Routing su modello economico per il volume ripetuto	~95% dei token (cache read) a tariffa minima
Calcoli e report consumavano token AI	Spostati su codice Python deterministico	Zero token e risultati esatti e ripetibili
Dati sensibili inviati al cloud	Modelli locali (Ollama) per banale e privato	Costo zero e nessun dato fuori dalla macchina
Contesto riletto troppo pesante	File operativi snelliti, heartbeat azzerato	Meno token rilette a ogni sessione
Salite a modelli più cari in automatico	Upgrade solo su decisione esplicita	Nessuna spesa premium silenziosa
Spesa AI opaca e prezzi errati nei log	Tracker sui transcript + saldo provider, tariffe corrette	Costi monitorati con numeri veri, ogni sera

Tabella 7.0 - La strategia costi come catena problema -> soluzione -> beneficio. È la sintesi operativa di ciò che i paragrafi tecnici seguenti documentano con i numeri. [verificato]

Approfondimento tecnico: dall'analisi dei token ai risultati

Le sottosezioni seguenti documentano in dettaglio la strategia di controllo dei costi con dati reali estratti dagli strumenti del sistema (lo script di monitoraggio della spesa è il log del saldo del provider economico). I numeri sono **misurati**, non stimati: dove un valore proviene direttamente dal tracker o dal log è marcato [verificato]; dove è una proiezione a tariffa su stesso volume è indicato come tale.

7.1 Analisi dei token: dove finisce davvero il costo

La prima scoperta, misurando i transcript reali, è che il costo di un agente con memoria non è dominato dalla generazione (token di output) ma dalla **rilettura del contesto** a ogni turno. Ogni messaggio rilegge cronologia, memorie e istruzioni: sono token di input, e nei modelli premium anche solo riproporli costa. Nel periodo osservato (2-19 giugno 2026, 18 giorni) il sistema ha registrato **3.864,3 milioni di token** totali, di cui **3.659,1 milioni in cache read**: il **~95%** dei token è riletture di contesto, non testo nuovo. I valori sono uno snapshot al 19 giugno 2026 e crescono con l'uso; le proporzioni e i rapporti tra modelli, invece, restano stabili. [verificato]

È questa la ragione tecnica per cui un modello premium usato in continuo diventa insostenibile: non perché risponda tanto, ma perché rilegge tanto. Tutte le leve di contenimento dei costi attaccano questo punto - o riducono i token riletti (contesto snello), o abbassano la tariffa per token (routing su modelli economici), o eliminano del tutto la chiamata (lavoro deterministico).

Voce di token (periodo 02-19/06)	Valore	Quota
Token totali processati	3.864,3 M	100%
di cui cache read (contesto riletto)	3.659,1 M	~95%
di cui input nuovo + output	~205,2 M	~5%

Tabella 8.1 - Composizione dei token su 18 giorni. Fonte: script di monitoraggio spesa (--totale). [verificato]

7.2 Monitoraggio dei token e della spesa

Il monitoraggio è interamente deterministico e a costo AI nullo. Uno script Python legge i transcript per-sessione (formato JSONL), somma i token per categoria (input, output, cache write a 5 minuti e a 1 ora, cache read) e applica le tariffe ufficiali per milione di token. Le tariffe non sono cablate alla cieca: un aggiornamento giornaliero le allinea ai listini pubblici dei provider, così i log di costo restano veritieri nel tempo. Lo script offre più viste: spesa reale del giorno, totale dall'inizio, dettaglio per giorno, e confronto comparativo su stesso volume. [verificato]

Nota metodologica. Durante la messa a punto del tracker è emerso che i prezzi DeepSeek in configurazione erano sbagliati (gonfiati fino a ~10x), il che falsava i log di costo. Sono stati corretti ai valori ufficiali verificati. È un esempio del principio caccia ai guasti silenziosi: un numero plausibile ma errato è più pericoloso di un errore evidente.

Accanto al tracker basato sui transcript, un secondo controllo legge il **saldo reale** del provider economico: è la prova del nove, perché confronta la stima a tariffa con il denaro effettivamente speso. Nel periodo 15-18 giugno il saldo DeepSeek è passato da **\$4,50 a \$4,41: \$0,09 di spesa reale in circa quattro giorni** di uso quotidiano sul default economico. [verificato]

Tariffe API (USD / milione di token)	Input	Output	Cache read
Fable 5	10,00	50,00	1,00
Claude Opus 4.8	5,00	25,00	0,50
Claude Sonnet 4.6	3,00	15,00	0,30
DeepSeek V4 Pro	0,435	0,87	0,003625
DeepSeek V4 Flash	0,14	0,28	0,0028

Tabella 8.2 - Tariffe per milione di token con cui il motore di tracciamento (spend_monitor.py) calcola la spesa; cambio indicativo USD->EUR 0,92. Verifica prezzi: 19/06/2026. I valori in tabella coincidono con la configurazione effettiva del tracker, alimentata giornalmente da models.dev (ultimo aggiornamento 18/06/2026) e ricontrollata sui listini ufficiali di prima parte di Anthropic e DeepSeek: input, output e cache read sono [verificato]. Per i provider il cui listino non pubblica una tariffa di cache read per token (es. Gemini, Grok) il tracker la modella e tali voci sono marcate [ricostruzione]; dove nemmeno una stima è attendibile, il dato non viene riportato.

Il divario di tariffa è la chiave economica: sul token di cache read - quello che domina il consumo - DeepSeek Flash costa **0,0028 \$/Mtok** contro i **0,50 \$/Mtok** di Opus, cioè circa **180 volte meno** sulla voce più pesante. È il motivo per cui spostare la routine sul default economico non è un risparmio marginale, ma di ordine di grandezza.

7.3 Simulazioni comparative: stesso volume, modelli diversi

La domanda decisiva non è quanto si è speso, ma **quanto sarebbe costato lo stesso identico lavoro** su ciascun modello. Il tracker risponde proiettando il volume reale di token (gli stessi 3.864,3 M su 18 giorni) sulle tariffe di ogni modello. La tabella seguente è la simulazione comparativa completa.

Modello	Costo a stesso volume (EUR, 18 giorni)	Rapporto vs DeepSeek Flash
Fable 5	8.114,54	~201x
Claude Opus 4.8	4.057,27	~100x
GPT-5.5	3.437,93	~85x
Claude Sonnet 4.6	2.434,36	~60x
GPT-5.4	1.718,96	~42x
Gemini 3.1 Pro	1.375,17	~34x
Gemini 3.5 Flash	1.031,38	~25x
Grok 4.3	949,84	~23x
Haiku 4.5	811,45	~20x
Gemini 3 Flash	343,79	~8,5x
Grok 4.1 Fast	215,81	~5,3x
DeepSeek V4 Pro	108,45	~2,7x
DeepSeek V4 Flash (default)	40,40	1x (riferimento)

Tabella 8.3 - Simulazione a parità di volume (3.864,3 M token, 02-19/06, snapshot al 19/06). Fonte: script di monitoraggio spesa (-totale). I rapporti sono calcolati sul costo in euro. [verificato]

Diagramma 7 - Confronto scenari di costo a parità di volume

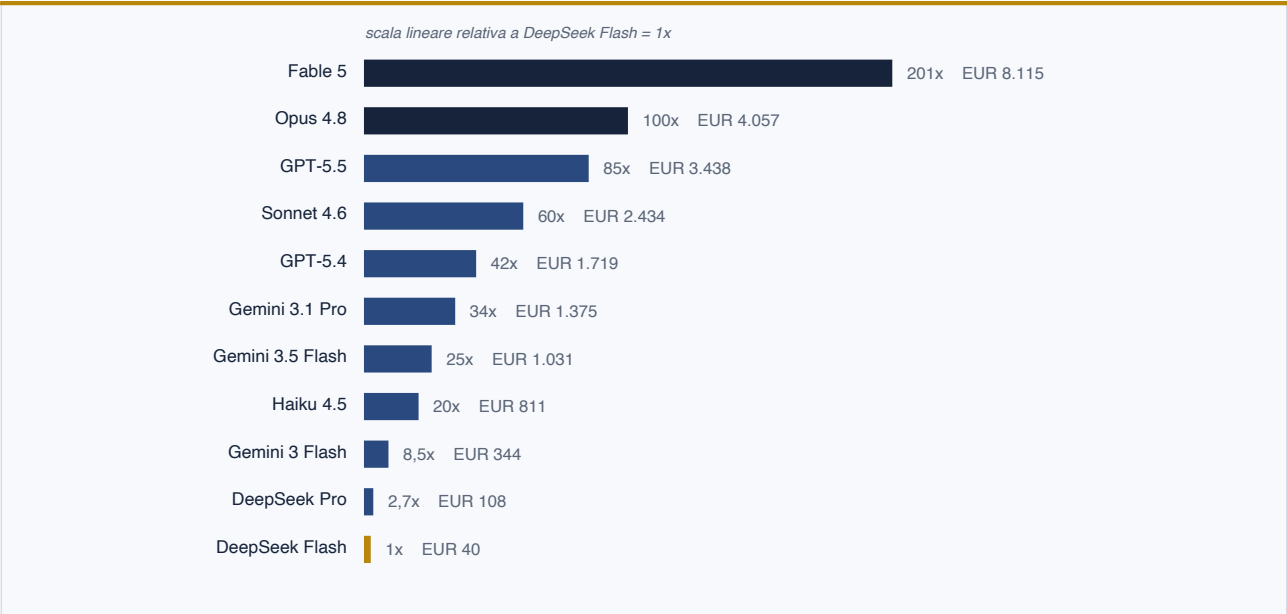


Diagramma 7 - Lo stesso carico di token costerebbe ~100x su Opus e ~201x su Fable 5 rispetto al default DeepSeek Flash. È la giustificazione quantitativa del routing economico. [verificato]

7.4 Strategia di routing economico

La strategia che ne discende è semplice da enunciare e misurabile nei suoi effetti: **il default è il modello più economico capace di svolgere il compito**; si sale di livello solo quando il compito lo richiede e solo con conferma esplicita; il banale e il sensibile restano in locale a costo zero; il lavoro deterministico non tocca affatto l'AI. In termini di token, la strategia lavora su tre assi: ridurre i token riletti (contesto snello, memoria curata invece di cronologia grezza), abbassare la tariffa per token (default economico), azzerare le chiamate dove non servono (Python deterministico per finanza, report, backup). È il punto che eleva il costo da dettaglio operativo a **vincolo architetturale**: in un agente sempre acceso la spesa va decisa prima della chiamata, in base al tipo di compito, non subito a posteriori. I meccanismi concreti che la realizzano - routing che evita le chiamate inutili, blocco del dispatch a pagamento, lavoro deterministico fuori dall'AI, modelli locali, premium solo on-demand - sono il tema del capitolo 8.

7.5 Risultati ottenuti

Va detto con onestà, perché il documento è marcato per livello di certezza. Nel periodo misurato la spesa **reale** registrata dal runtime è stata di circa **4.011 EUR** (snapshot al 19/06, in crescita), quasi interamente dovuta all'uso premium on-demand di Claude Opus (incluse le sessioni di sviluppo intensivo, come la redazione di questo stesso documento): **non è il consumo del default architetturale**, ma il costo di scelte premium esplicite. Il consumo del default economico, misurato sul saldo reale del provider, è dell'ordine di **pochi centesimi al giorno** (\$0,09 in quattro giorni). [verificato]

Il risultato chiave non è quindi una singola cifra di spesa, ma la **distanza tra gli scenari**: lo stesso volume di lavoro che su un modello premium costerebbe migliaia di euro, instradato sul default economico ne costa qualche decina; e il lavoro deterministico - la maggior parte del modulo Portfolio, dei report e dei backup - non costa nulla in token. Il routing non è una micro-ottimizzazione: è ciò che rende l'intero ecosistema economicamente sostenibile come sistema sempre acceso. [verificato]

7.6 Confronto diretto: reale vs simulato (Claude, DeepSeek, locale)

La tabella seguente mette a confronto, sullo **stesso identico volume di token realmente consumato**, il costo reale osservato e il costo che lo stesso lavoro avrebbe avuto su ciascuna famiglia di modelli. È il confronto che riassume tutta la strategia in una riga.

Scenario (volume reale: 3.864,3 M token, 02-19/06, snapshot al 19/06)	Modello	Costo del periodo	Livello
Costo reale osservato (mix effettivo, in prevalenza Opus on-demand)	runtime	~4.011,28 EUR	[verificato]
Simulato - tutto su Claude Opus 4.8	Opus 4.8	4.057,27 EUR	[verificato]
Simulato - tutto su Claude Fable 5	Fable 5	8.114,54 EUR	[verificato]
Simulato - tutto su DeepSeek V4 Pro	DeepSeek Pro	108,45 EUR	[verificato]
Simulato - tutto su DeepSeek V4 Flash (default adottato)	DeepSeek Flash	40,40 EUR	[verificato]
Simulato - tutto su modelli locali (Ollama)	qwen3:8b locale	0,00 EUR (gratis)	[verificato]
Spesa reale del default economico (saldo provider)	DeepSeek Flash	~0,09 USD / 4 giorni	[verificato]

Tabella 8.5 - Costo reale osservato a confronto con il costo simulato su Claude, DeepSeek e modelli locali, a parità di volume. Fonte: script di monitoraggio spesa (--totale) + log saldo provider. [verificato]

8. Tecniche di riduzione costi e token

Se il capitolo precedente misura quanto costa il sistema, questo spiega **come quel costo viene tenuto basso**. Non è un elenco di accorgimenti isolati, ma un insieme coerente di scelte progettuali, ciascuna mirata a un meccanismo preciso di spesa: o riduce i token consumati, o abbassa la tariffa pagata per token, o azzera del tutto la chiamata. È uno dei capitoli portanti del documento, perché in un agente sempre acceso il controllo dei costi non è un dettaglio operativo ma un vincolo di progetto. [verificato]

Routing locale prima del cloud

La prima leva è non chiamare affatto un modello a pagamento quando non serve. Il router a plugin (eseguito *before_dispatch*) intercetta ogni richiesta prima dell'invio al cloud: se può essere risolta in locale - un comando, una risposta di servizio, un compito a basso valore o sensibile - la chiamata cloud non parte e i token consumati sono zero. Quando il plugin marca l'evento come gestito (*handled:true*), il dispatch verso il modello a pagamento è bloccato alla radice. [verificato]

Modalità gratis on

Il comando **gratis on** forza tutto il traffico su un modello locale via Ollama (qwen3:8b quantizzato), a costo zero per token: da quel momento si dialoga solo col modello locale e il provider cloud non si attiva. È pensato per le conversazioni che non richiedono la qualità del cloud. **gratis off** ripristina il default economico. In questa modalità le integrazioni che dipendono da tool cloud (posta, casa, portafoglio) non sono disponibili, perché il locale espone solo un sottoinsieme di strumenti: è un compromesso esplicito tra costo zero e funzionalità. [verificato]

Modalità gratis libero

La variante **gratis libero** entra direttamente con una versione locale alternativa dello stesso modello, con filtri di risposta meno restrittivi. Resta interamente locale e a costo zero: serve a coprire i casi in cui il modello standard rifiuterebbe richieste legittime, senza ricorrere al cloud. Il principio economico è identico - nessun token a pagamento - e cambia solo la variante di modello caricata. [ricostruzione]

Politica dei modelli: il premium non è mai il default

- **Claude mai come default**: il modello più costoso (fino a ~100-200x il default sulla cache read) non viene mai scelto automaticamente. Eliminare il default premium toglie alla radice la voce di spesa più pesante. [verificato]
- **DeepSeek Pro solo con conferma**: anche la salita al modello economico più capace richiede una conferma esplicita. Si evita l'escalation silenziosa che moltiplicherebbe la tariffa senza un reale bisogno. [verificato]
- **Modelli locali via Ollama**: OCR, lettura di documenti sensibili e richieste a basso valore girano in locale, senza tariffa per token e senza inviare dati all'esterno; coprono anche da fallback se il cloud non è raggiungibile. [verificato]

Deterministico invece di AI

- **Separazione tra AI e logica Python**: ciò che ha una risposta esatta e ripetibile è codice, non un modello. Valore, P&L, ranking, scoring, rischio e Monte Carlo sono Python: passarli da un'AI costerebbe token e introdurrebbe errori. [verificato]
- **Report deterministici**: i report giornalieri e settimanali hanno layout fisso e dati reali, generati da codice. Zero token per un output che non richiede giudizio linguistico. [verificato]
- **Automazioni senza AI**: pulizia, conteggi, invii schedulati e pricing girano su launchd/cron come codice. Il lavoro ripetitivo non tocca un modello a pagamento. [verificato]
- **Backup senza AI**: il backup è un'operazione meccanica e critica; affidarla a codice deterministico la rende affidabile e a costo zero in token. [verificato]

Cache e gestione intelligente del contesto

La voce di spesa dominante non è ciò che l'agente scrive, ma ciò che **rilegge** a ogni turno: circa il 95% dei token è riletture di contesto (cache read). Per questo la leva quantitativamente più efficace è ridurre e governare il contesto. [verificato]

- **AGENTS.md ottimizzato**: istruzioni di sistema compatte; gli approfondimenti pesanti (note dispositivi, cataloghi comandi) si aprono solo a richiesta. Meno testo di sistema = meno token riletti a ogni messaggio. [verificato]
- **Contesto snello**: cronologia compressa, memoria curata al posto del log grezzo, file di startup non riletti se già in contesto. Ogni token tolto dal contesto si risparmia a ogni turno successivo. [verificato]
- **Eliminazione delle chiamate inutili**: gli heartbeat rispondono con un marcatore quando non c'è nulla da fare e i controlli periodici sono accorpati; comandi e richieste già gestite non raggiungono un modello a pagamento. [verificato]
- **Monitoraggio della cache read**: il tracker misura esplicitamente i token di cache read, rendendo visibile la causa reale del costo e permettendo di agire su quella, non su quella percepita. [verificato]

Impatto economico delle scelte progettuali

Queste tecniche non sono micro-ottimizzazioni: ne va dell'ordine di grandezza della spesa. Come mostra il capitolo 7, lo stesso volume di lavoro che su un modello premium costerebbe migliaia di euro, instradato sul default economico ne costa qualche decina, e sul locale costa zero. Il filo conduttore è uno solo: **spendere token solo dove serve davvero giudizio o linguaggio**, alla tariffa più bassa compatibile col compito. Tutto ciò che ha una risposta esatta esce dall'AI; tutto ciò che è banale o sensibile resta in locale; tutto ciò che richiede il premium lo ottiene solo su decisione esplicita. È questo insieme di scelte, non un singolo trucco, a rendere economicamente sostenibile un agente sempre acceso. [verificato]

HETEPI AI · ECOSISTEMA OPENCLAW

9. Cosa è deterministico e cosa è AI

Una delle decisioni più importanti dell'intero ecosistema è la linea di confine tra ciò che fa un algoritmo deterministico e ciò che fa un modello AI. Il criterio è netto: se un compito ha una risposta esatta e ripetibile, non deve passare dall'AI. L'AI è riservata a ciò che richiede giudizio, linguaggio o interpretazione.

La divisione

Componente	Tipo	Motivo della scelta
Calcoli finanziari (valore, P&L)	Deterministico	Risposta esatta: l'AI introdurrebbe errori e costo inutile
Ranking / scoring titoli	Deterministico	Formula ripetibile e verificabile
Alert di prezzo	Deterministico	Soglie precise, niente ambiguità
Report giornalieri/settimanali	Deterministico	Layout fisso, dati reali, zero token
Backup	Deterministico	Operazione meccanica e critica, dev'essere affidabile
Pricing / parsing dati di mercato	Deterministico	Numeri esatti; il parsing va controllato, non interpretato
Test	Deterministico	Esito booleano, nessun giudizio
Ragionamento e supporto decisionale	AI	Richiede valutazione di contesto
Scrittura e sintesi	AI	Linguaggio naturale, tono, leggibilità
Audit del codice / del sistema	AI	Riconoscere pattern e incoerenze
Interpretazione di richieste vaghe	AI	Capire l'intento, non solo le parole

Tabella 9 - La frontiera deterministico/AI. Il grosso del modulo Portfolio è deterministico; l'AI interviene su linguaggio, giudizio e interpretazione.

Questa divisione ha tre benefici diretti: **affidabilità** (i numeri non allucinano), **costo** (i calcoli sono gratis) e **verificabilità** (un risultato deterministico si ricontrolla con un test, una risposta AI no). È anche il motivo per cui il modulo Portfolio, pur essendo intelligente, è quasi interamente codice Python e non chiamate a un modello.

10. Memoria e personalizzazione

L'agente non riparte da zero a ogni conversazione: ha una memoria persistente su file, organizzata su due livelli. È ciò che lo rende personale e non intercambiabile con un assistente generico.

I due livelli di memoria

- **Note giornaliere** (memory/AAAA-MM-GG.md): log grezzo di cosa è successo, scritto giorno per giorno.
- **Memoria a lungo termine** (MEMORY.md + file curati per tema): l'essenza distillata - preferenze, regole, decisioni, storia del progetto - indicizzata e richiamabile.

La memoria curata è suddivisa per tipo: profilo utente, feedback (regole di comportamento imparate), note di progetto, riferimenti a sistemi esterni. Ogni voce ha una descrizione che ne facilita il richiamo quando pertinente. [verificato]

Cosa l'agente sa di me

- **Identità e lingua**: utente Andrea, risposte sempre in italiano.
- **Stile**: risposte brevi, dirette, senza filler e senza domande di onboarding superflue.
- **Precisione**: pretendo dati reali e verificabili; output disordinati o con dati inaffidabili vengono respinti.
- **Formato**: niente righe separatrici, niente tabelle markdown su Telegram (si rompono); testo umano che interpreta i dati a parole.
- **Regola voce**: a un vocale rispondo con un vocale; a un testo con un testo.
- **Workflow ricorrenti**: audit prima di modificare, un cambiamento per volta, conferma prima delle azioni che cambiano stato.

Queste preferenze non sono decorative: modificano attivamente le risposte. Lo stile asciutto, l'assenza di separatori, la prosa che spiega i numeri, la scelta del canale di risposta - tutto discende da regole salvate in memoria e applicate a ogni interazione.

Nota metodologica. La memoria è un fotogramma nel tempo: registra ciò che era vero quando è stata scritta. Prima di agire su un dato di memoria, l'agente verifica che sia ancora valido leggendo lo stato attuale del sistema; se diverge, prevale ciò che osserva ora e la memoria viene aggiornata.

HETEPI AI · ECOSISTEMA OPENCLAW

11. Integrazioni e sistemi controllati

L'agente non vive in isolamento: controlla o gestisce un insieme di sistemi reali. È questo che lo rende operativo e non solo conversazionale.

Sistema controllato	Tecnologia	Stato	Cosa permette di fare
Bot Telegram	OpenClaw + API Telegram	Attivo	Interfaccia unica di comando e risposta
Portfolio Intelligence	Python deterministico + bot	Attivo	Analisi, scoring, rischio, report multi-utente
Google Workspace	CLI dedicata (Gmail, Calendar, Drive)	Attivo	Posta, calendario, documenti, contatti
Voce (STT)	Whisper locale	Attivo	Trascrizione dei vocali in ingresso
Voce (TTS)	Piper locale / ElevenLabs	Attivo	Risposte vocali (Piper gratis, fallback)
TV (salone, camera)	Samsung WebSocket + WoL	Attivo	Accensione, tasti, canali, YouTube
TV (Hisense)	AirPlay 2 + WoL	Attivo	Controllo camera secondaria
Robot aspirapolvere	Cloud Dreame	Attivo	Avvio/stop, pulizia per stanza, stato
Luci Philips Hue	API bridge locale	Attivo	On/off, luminosità, colore, scene
Backup	Script + cifratura	Attivo	Backup settimanale cifrato del workspace
Automazioni	launchd (16 servizi)	Attivo	Report, pulizia, monitor, reminder
Media / webcam	ffmpeg	Attivo	Foto/video on-demand, poi cancellati

Tabella 10 - Sistemi sotto controllo dell'agente. Le schede operative dei dispositivi sono tenute in file dedicati e aperte solo al bisogno, per non appesantire il contesto.

Un dettaglio di design ricorrente: le istruzioni operative pesanti (come comandare una specifica TV o il robot) non sono sempre in memoria, ma in schede separate lette solo quando servono. Così l'indice resta leggero e ogni messaggio non paga il costo di contesto di informazioni che il più delle volte non servono. [verificato]

12. Portfolio Intelligence come caso studio avanzato

Portfolio Intelligence non è il protagonista di questo documento, ma è la prova più concreta di cosa l'agente abbia abilitato: la costruzione, in poche settimane, di un prodotto software complesso, multi-utente e quasi interamente deterministico. È il caso studio che dimostra che Hetepi non è solo un bot Telegram.

Cosa fa

- **Architettura:** un nucleo di librerie Python (calcolo, pricing, scoring, rischio, simulazione) separato dal bot Telegram che fa da interfaccia; lo stato persiste su uno store centralizzato (SQLite in modalità WAL) con profili per-utente, e i servizi schedulati (launchd) producono report e alert indipendentemente dalla chat. [verificato]
- **Bot multi-utente:** un unico bot riconosce l'utente dal chat_id e tiene profili separati (holdings, stato, consegne, watchlist) per più persone, con un registro che generalizza gli stessi strumenti su portafogli diversi senza hard-coding. [verificato]
- **Motore di analisi:** valore live, performance per orizzonte (giorno/settimana/mese su NAV reale), composizione, valute e dividendi; pricing da fonti di mercato con parsing controllato (virgola/punto, ISIN completi, DoH) e allineamento al broker (Tradegate / Trade Republic, bid vs last). [verificato]
- **Gestione del rischio:** VaR, beta, stress test su shock storici, correlazioni reali tra titoli e analisi di profondità; metriche deterministiche, ricontrollabili con i test. [verificato]
- **Monte Carlo:** simulazione probabilistica degli scenari futuri con drift prudente e block bootstrap, scenari per-asset invece di un singolo modello semplicistico, per proiezioni più oneste. [verificato]
- **Alert:** soglie di prezzo per-titolo, brief giornaliero e settimanale, tutti schedulati e a costo AI nullo; gli alert scattano su regole precise, senza ambiguità. [verificato]
- **Motore di consigli:** scoring dei titoli con scenari dinamici; il backtest walk-forward ha mostrato che lo score predice la sicurezza (drawdown) più del rendimento - e quindi usato come selettore prudente, non come promessa di guadagno. La dashboard usa grafici dark-mode e transizioni in-place stile app, con linguaggio da analista sempre su dati reali. [verificato]

Componente deterministica vs AI nel modulo

Il modulo applica in piccolo la frontiera del capitolo 9: ciò che ha una risposta esatta resta in Python, l'AI agisce solo sulla forma.

Componente del modulo	Tipo	Nota
Valore, P&L, performance per orizzonte	Deterministico	Calcolo esatto su NAV reale
Scoring / motore di consigli	Deterministico	Formula ripetibile e verificabile
Rischio (VaR, beta, stress, correlazioni)	Deterministico	Metriche su serie storiche reali
Monte Carlo	Deterministico	Drift prudente + block bootstrap, seed controllato
Alert di prezzo e report schedulati	Deterministico	Soglie precise, layout fisso, zero token
Stesura in prosa del report / leggibilità	AI	Solo linguaggio e sintesi, mai i numeri

Tabella 11.1 - Nel modulo Portfolio i calcoli sono deterministici (zero token); l'AI agisce solo sulla leggibilità dell'output.

Perché è un buon banco di prova

Il modulo incarna i principi dell'ecosistema: calcoli deterministici (zero token), dati reali e verificati (prezzi di mercato, NAV), generalizzazione su portafogli diversi senza hard-coding e robustezza presidiata da una suite di test (oltre 170 controlli smoke); l'AI interviene solo nel rendere leggibile l'output, mai nei numeri. La sua evoluzione è documentata da decine di milestone datate (giugno 2026): in piccolo, lo stesso metodo applicato a tutto l'ecosistema. [verificato]

HETEPI AI · ECOSISTEMA OPENCLAW

13. Problemi affrontati e soluzioni

Un sistema reale si misura dai problemi che ha attraversato. Questa tabella raccoglie i nodi più significativi: causa, soluzione, risultato.

Problema	Causa	Soluzione adottata	Risultato
Costi AI fuori controllo	Modello premium su contesto enorme riletto ogni turno	Routing: default economico + locale + premium on-demand	Spesa riportata a pochi euro/giorno [stima]
Output troppo lunghi	Risposte prolisse e con riempitivi	Regole di stile in memoria: brevità, niente filler	Output executive, leggibili
Monte Carlo poco solido	Modello di simulazione semplicistico	Drift prudente + block bootstrap, scenari per-asset	Proiezioni più oneste
Dati finanziari non allineati	Differenze tra fonti e broker	Allineamento a Tradegate/Trade Republic, bid vs last	Valori coerenti col broker
Prezzi live sbagliati	Parsing numeri (virgola/punto) e DNS	Fix parsing + DoH, ISIN completi	Prezzi corretti e stabili
Generalizzazione portafogli	Logica tarata su un solo profilo	Astrazione multi-utente registry-driven	Stessi strumenti per più utenti
Prezzi DeepSeek errati in config	Valori gonfiati fino a ~10x	Correzione ai prezzi ufficiali verificati	Tracker spesa attendibile
Bot fermi / daemon instabili	Processi non supervisionati	launchd con KeepAlive	Servizi sempre attivi
Test rossi preesistenti	Debito tecnico su alcuni footer	Isolati e tracciati come debito separato	Resto della suite verde
UI sporca	Layout incoerente, separatori	Pass qualità output, transizioni in-place	Presentazione pulita e coerente
Media privati conservati	Nessuna pulizia automatica	Automazione che cancella i media dopo l'uso	Privacy garantita

Tabella 11 - Problema -> causa -> soluzione -> risultato. Molte voci derivano da audit espliciti richiesti durante lo sviluppo.

14. Metodo di sviluppo

Il sistema non è cresciuto a caso: dietro c'è un metodo ripetibile, applicato a ogni modifica non banale. È esso stesso parte dell'architettura, perché determina la qualità e la stabilità del risultato.

I principi del metodo

- **Audit prima di modificare**: capire lo stato attuale prima di toccarlo.
- **Piccoli passi**: un cambiamento per volta, perimetro chiaro.
- **Test**: verifica automatica dove possibile.
- **Rendering reale**: controllare l'output sulla superficie vera (il bot), non in astratto.
- **Deploy solo dopo approvazione**: nessuna modifica esterna senza conferma.
- **Restart controllati**: un solo riavvio per volta, mai a catena (un restart a catena ha già causato un crash).
- **Log puliti e backup prima dei refactor**: poter sempre tornare indietro.

Il ciclo

Diagramma 5 - Il ciclo di sviluppo

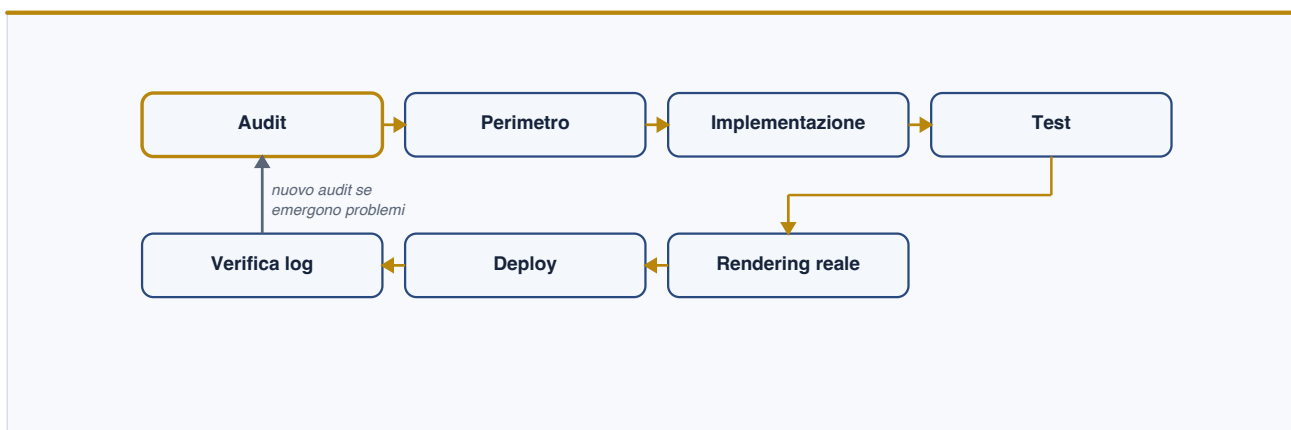


Diagramma 5 - Audit -> Perimetro -> Implementazione -> Test -> Rendering -> Deploy -> Verifica. La verifica può riaprire il ciclo.

Un corollario importante: non dire che **funziona** senza prova end-to-end sulla superficie reale. Gli errori più insidiosi sono quelli silenziosi - un fallback che resta verde ma degrada il dato, un None mascherato da un except troppo largo. Il metodo include la caccia attiva a questi guasti, perché un dato sbagliato ma plausibile è peggio di un errore evidente.

15. Risultati raggiunti

La trasformazione si legge meglio per contrasto. Si parte da un chatbot che risponde con testo generico, un solo modello costoso, nessuna memoria, nessun backup e azioni svolte una a una a mano. Si arriva a un **agente operativo** che esegue azioni reali, con memoria persistente e personalizzazione, una famiglia di modelli dai ruoli distinti, routing che tiene la spesa sotto controllo, 16 automazioni schedate, backup settimanale cifrato e un modulo finanziario multi-utente al posto delle analisi manuali. [verificato]

Il risultato non è una somma di funzioni, ma un cambiamento di natura: da strumento che produce testo a sistema che percepisce, decide, agisce e rende conto - entro vincoli di costo e privacy stabiliti fin dall'inizio.

Sintesi quantitativa

- **16 automazioni schedate** via launchd (report, backup, pulizia, monitoraggio, reminder). [verificato]
- **Famiglia di modelli a ruoli distinti**: DeepSeek Flash come default, DeepSeek Pro su conferma, Claude solo a richiesta, Ollama (qwen3:8b e OCR) in locale. [verificato]
- **Routing economico**: lo stesso carico di token costerebbe circa 100x su Opus e 201x su Fable 5 rispetto al default Flash (cfr. Diagramma 7). [verificato]
- **Memoria persistente**: note giornaliere più memoria curata per tema, lette e aggiornate a ogni sessione. [verificato]
- **Backup settimanale cifrato** del workspace e pulizia automatica dei media in ingresso. [verificato]
- **Portfolio Intelligence** multi-utente in produzione, con suite di test (170+ smoke check). [verificato]

Dai risultati agli obiettivi iniziali

Letti rispetto all'obiettivo di partenza - trasformare un assistente conversazionale in un agente operativo affidabile, economico e rispettoso della privacy - i risultati chiudono il cerchio. L'operatività è data dalle azioni reali su casa, posta e file; l'affidabilità dalla separazione tra logica deterministica in Python e uso mirato dell'AI; il controllo dei costi dal routing che mantiene il default sul modello più economico e confina i modelli cari alle sole richieste esplicite; la privacy dall'esecuzione in locale di ciò che è banale o sensibile. Ogni beneficio architetturale descritto nei capitoli precedenti trova qui il suo riscontro misurabile, senza che nessuna delle decisioni progettuali adottate sia stata contraddetta dall'uso reale del sistema.

16. Architettura attuale in una pagina

Componenti principali

Area	Componenti
Ingresso	Bot Telegram privato (un solo chat_id autorizzato)
Orchestrazione	OpenClaw gateway (launchd) + plugin di routing
Modelli	DeepSeek Flash (default) - DeepSeek Pro (on-demand) - Claude (a richiesta) - Ollama qwen3:8b/abliterated/OCR (locale)
Voce	Whisper (STT locale) - Piper / ElevenLabs (TTS)
Tool / azioni	Google Workspace - TV - robot - luci Hue - webcam - file
Memoria	Note giornaliere + memoria curata (MEMORY.md e file per tema)
Storage / stato	openclaw.json - catalog provider - file di stato modalità - holdings per-utente
Automazioni	16 servizi launchd: report, backup, pulizia, monitor, reminder
Backup	Backup settimanale cifrato del workspace
Modulo verticale	Portfolio Intelligence (Python deterministico + bot multi-utente)

Tabella 13 - L'intero sistema in una vista sintetica.

17. Stato attuale e prossimi passi

Cosa funziona oggi

- Agente Telegram operativo, con routing dei modelli e modalità a costo zero. [verificato]
- Memoria persistente e personalizzazione attiva. [verificato]
- 16 automazioni schedate, backup cifrato, tracciamento spesa. [verificato]
- Portfolio Intelligence multi-utente in produzione, con suite di test. [verificato]
- Controllo casa (TV, robot, luci) e Google Workspace. [verificato]

Cosa è stabile e cosa resta da migliorare

- **Stabile:** gateway, routing, automazioni deterministiche, backup.
- **Debito noto:** alcuni test sui footer dei bottoni, isolati e tracciati come debito separato.
- **Da rifinire:** copertura dei tempi di risposta cloud (oggi non misurati), e ulteriore snellimento del contesto.

Direzioni future

- **Portfolio Intelligence v2:** evoluzione del modulo verticale.
- **Modelli locali più potenti:** man mano che l'hardware/Ollama lo consentono, spostare di più sul locale.
- **Agenti specializzati:** sotto-agenti dedicati a compiti specifici.
- **Disaster recovery:** procedura di ripristino completa oltre al backup cifrato.

Replicabilità del framework

Una domanda naturale è se questo ecosistema sia un caso isolato o un **modello replicabile**. L'architettura è stata pensata per essere riusata, perché i suoi componenti sono **disaccoppiati e sostituibili**: l'interfaccia (qui Telegram) è un canale tra molti; il router è un plugin isolato; i modelli sono intercambiabili dietro una configurazione dichiarativa; la memoria è fatta di file leggibili; le automazioni sono servizi del sistema operativo, indipendenti dall'AI. Per riusare l'architettura su un altro dominio bastano tre mosse: ridefinire il confine deterministico/AI, configurare i modelli e le loro tariffe, e scrivere le automazioni deterministiche specifiche. Il resto - gateway, routing, memoria, fallback - resta invariato. Gli scenari aziendali concreti, con problema, architettura e benefici per ciascuno, sono trattati nel capitolo 18.

Diagramma 8 - Il telaio riusabile del framework

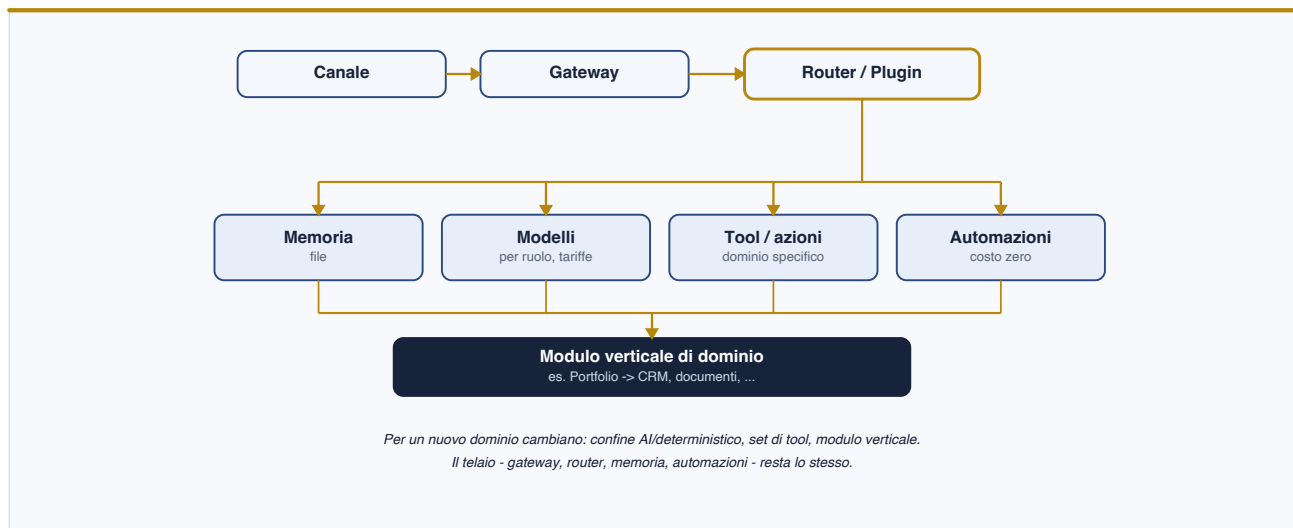


Diagramma 8 - I componenti chiari e sostituibili rendono l'architettura un modello replicabile, non una soluzione su misura.

18. Replicabilità in azienda

Questo ecosistema nasce per un uso personale, ma la sua architettura non dipende dal caso d'uso specifico. La domanda di questo capitolo non è *se* sia replicabile - il capitolo 17 ha già mostrato che il telaio è riusabile - bensì *come* adattarlo, caso per caso, a un'azienda o a un professionista. Per ogni scenario sono indicati: il problema aziendale, l'architettura proposta, quali componenti restano locali, quali possono usare AI cloud, e i benefici attesi, con attenzione particolare alla riduzione dei costi e alla riduzione della dipendenza da fornitori AI esterni.

Il principio di adattamento

Adattare il framework a un contesto aziendale non significa riscriverlo. Significa tre cose: ridefinire il confine deterministico/AI per il dominio (cosa deve essere un calcolo esatto e cosa può essere una sintesi del modello), decidere quali dati e quali compiti restano in casa e quali si delegano a un modello cloud, e configurare modelli e tariffe in funzione del budget. Il telaio - ingresso unico, router, memoria su file, automazioni di sistema, fallback - resta invariato; cambiano i tool e il modulo verticale di dominio.

Quadro d'insieme

Caso d'uso	Problema da risolvere	Resta locale	Può usare AI cloud
Assistente operativo aziendale	Eeguire azioni reali sui sistemi, non solo rispondere	Azioni, integrazioni, dati operativi, memoria	Comprensione richieste, sintesi
Help desk interno	Prima linea di supporto a dipendenti	Base di conoscenza, log ticket, routing	Riformulazione e risposta in linguaggio naturale
Gestione documentale	Acquisire e ordinare grandi volumi di documenti	Acquisizione/OCR, classificazione, archivio	Estrazione semantica, riassunto
Knowledge base aziendale	Trovare informazioni interne senza esporle	Indice e documenti, citazione delle fonti	Risposta con citazione, sintesi
Automazioni amministrative	Compiti ripetitivi a basso valore	Regole, riconciliazioni, invii schedulati	Solo casi ambigui, interpretazione
Reportistica automatica	Produrre report periodici senza lavoro manuale	Estrazione dati, calcoli, grafici, invio	Stesura del commento narrativo
Monitoraggio operativo	Accorgersi dei problemi prima dei clienti	Sonde, soglie, alert, supervisione servizi	Spiegazione dell'anomalia, priorità
Studi professionali	Pratiche e clienti separati, dati sensibili	Profili cliente, scadenziario, calcoli, archivio	Bozze, ricerche, riassunti
PMI	Un solo assistente per molte funzioni, budget ridotto	Tutto il deterministico, dati, memoria	Routine su modello economico, sintesi
Consulenza finanziaria	Analisi su portafogli e dati di mercato affidabili	Pricing, rischio, scoring, Monte Carlo, profili	Commento, scenari narrativi, sintesi
Industria e manutenzione	Anticipare guasti e ordinare gli interventi	Sensori, soglie, storico, schede macchina	Spiegazione anomalie, istruzioni operative

Tabella 17.1 - Gli undici scenari a colpo d'occhio: cosa resta in casa e cosa può essere delegato a un modello cloud. I cinque casi più rappresentativi sono approfonditi nei paragrafi seguenti. [ricostruzione]

Cinque casi aziendali approfonditi

I cinque casi che seguono sono trattati in modo più completo, lungo sette dimensioni costanti: problema, architettura applicabile, benefici, riduzione della dipendenza da fornitori AI esterni, vantaggi del locale, controllo dei dati e controllo dei costi. Sono i contesti in cui il framework mostra meglio il proprio valore: dominio verticale, dati sensibili e budget da governare.

Studio professionale

- **Ambiti coperti:** gestione documentale, email, agenda e scadenziario, automazioni amministrative.
- **Problema:** studi legali, notarili e commercialisti gestiscono pratiche e clienti distinti, con dati riservati e scadenze inderogabili; il tempo si perde tra archivio, posta, agenda e ricerca documentale.
- **Architettura applicabile:** profili separati per cliente, scadenziario e calcoli deterministici, archivio e indice locali; connettori a posta e calendario per smistare e promemoriare; l'AI produce bozze, ricerche e riassunti solo su materiale già selezionato.
- **Benefici:** più pratiche gestite in meno tempo, nessuna scadenza persa, bozze pronte da revisionare.
- **Riduzione dipendenza da fornitori AI:** il valore sta nell'archivio e nei modelli di calcolo locali, non nel modello cloud, che resta intercambiabile.
- **Vantaggi del locale:** il materiale più sensibile (atti, posizioni dei clienti) può essere trattato da un modello locale, senza uscire dallo studio.
- **Controllo dei dati:** i fascicoli restano in casa; al modello cloud arriva solo il brano necessario a una specifica richiesta.
- **Controllo dei costi:** la routine gira su un modello economico; il premium si usa solo su decisione esplicita, con spesa proporzionata all'uso.

PMI

- **Ambiti coperti:** procedure operative, knowledge base interna, assistenza ai dipendenti, reportistica.
- **Problema:** una piccola o media impresa deve coprire molte funzioni (procedure, supporto interno, report) con budget AI ridotto e poche competenze tecniche dedicate.
- **Architettura applicabile:** un assistente unico con router per intento; knowledge base locale di procedure e policy con citazione delle fonti; help desk interno che deflette le domande ricorrenti; automazioni di sistema per report e riconciliazioni; l'AI solo su interpretazione e sintesi.
- **Benefici:** una sola soluzione per più esigenze, prima risposta immediata ai dipendenti, report puntuali senza lavoro manuale.
- **Riduzione dipendenza da fornitori AI:** i modelli sono sostituibili; il cuore operativo (knowledge base, automazioni, dati) funziona anche senza cloud.
- **Vantaggi del locale:** procedure e dati interni restano in azienda; il sensibile può girare su modello locale.
- **Controllo dei dati:** nessun documento interno esce dall'azienda se non per le sole sintesi che lo richiedono.
- **Controllo dei costi:** routine su modello economico, premium su conferma; il grosso del lavoro è codice deterministico a costo AI nullo.

Ufficio tecnico

- **Ambiti coperti:** documentazione tecnica, procedure e normative, supporto operativo al tecnico.
- **Problema:** un ufficio tecnico (progettazione, ingegneria, edilizia) gestisce capitolati, normative, computi e revisioni; ritrovare la versione giusta di un documento o una clausola normativa fa perdere tempo.
- **Architettura applicabile:** knowledge base locale di norme e progetti con citazione obbligatoria delle fonti; computi e verifiche numeriche deterministici; l'AI per estrazione semantica, confronto fra versioni e stesura di bozze tecniche.
- **Benefici:** documentazione sempre ricercabile e tracciabile, computi affidabili, meno errori di versione.

- **Riduzione dipendenza da fornitori AI:** l'indice e i calcoli sono in casa; il modello aggiunge solo linguaggio e sintesi, ed è rimpiazzabile.
- **Vantaggi del locale:** progetti e capitolati riservati non lasciano l'ufficio; l'indicizzazione e l'OCR sono locali.
- **Controllo dei dati:** al modello arrivano solo i brani necessari; i numeri non vengono mai prodotti dall'AI ma dal codice.
- **Controllo dei costi:** i computi non consumano token; l'AI interviene a campione, con spesa che cresce solo con l'uso reale.

Family Office

- **Ambiti coperti:** monitoraggio degli investimenti, reportistica periodica, analisi di rischio e scenari.
- **Problema:** un family office monitora portafogli e patrimoni producendo numeri affidabili e commenti chiari, senza che l'AI inventi cifre o esponga le posizioni dei clienti.
- **Architettura applicabile:** generalizzazione del modulo Portfolio Intelligence - pricing, rischio (VaR, beta, stress), scoring e Monte Carlo deterministici per cliente, con alert e report schedulati; l'AI scrive solo commento e scenari sui numeri già calcolati.
- **Benefici:** numeri sempre affidabili (mai prodotti dall'AI), report e monitoraggio puntuali, scenari illustrativi su dati verificati.
- **Riduzione dipendenza da fornitori AI:** il valore sta nei modelli di calcolo locali; il modello cloud è un commentatore sostituibile, non il motore.
- **Vantaggi del locale:** profili e posizioni dei clienti restano in casa; il calcolo del rischio non dipende da servizi esterni.
- **Controllo dei dati:** le posizioni patrimoniali non vengono esposte; al modello arrivano solo aggregati e numeri già calcolati.
- **Controllo dei costi:** il pesante è deterministico (zero token); l'AI scrive poche righe, con costo basso e prevedibile.

Operations aziendali

- **Ambiti coperti:** workflow operativi, automazioni, riduzione delle attività ripetitive.
- **Problema:** nelle operations molte attività sono ripetitive e i problemi vanno notati prima dei clienti; il sapere è sparso tra persone e file e spesso chi cerca un'informazione la ricostruisce da zero.
- **Architettura applicabile:** workflow e attività ripetitive trasformati in automazioni deterministiche schedulate; memoria curata di documenti e note con richiamo e citazione delle fonti; sonde, soglie e supervisione dei servizi deterministiche; l'AI per risposta con citazione, sintesi e spiegazione delle anomalie.
- **Benefici:** meno attività manuali ripetitive, sapere accessibile e tracciabile, rilevamento tempestivo dei problemi.
- **Riduzione dipendenza da fornitori AI:** la conoscenza e l'indice sono in casa e il monitoraggio funziona anche senza cloud; il modello è un livello aggiuntivo, non un single point of failure.
- **Vantaggi del locale:** i documenti interni non vengono esposti; il sensibile può restare su modello locale.
- **Controllo dei dati:** l'indice resta locale e al modello arrivano solo i brani necessari, sempre con fonte citata e verificabile.
- **Controllo dei costi:** alert e monitoraggio non usano AI; il costo cresce solo con l'uso reale delle risposte in linguaggio naturale.

19. Evoluzione delle decisioni tecniche

Un'architettura raramente nasce già nella forma definitiva: si assesta per tentativi, urtando contro limiti reali e correggendo la rotta. Questo capitolo non descrive lo stato attuale - lo fanno i precedenti - ma il **ragionamento dietro le scelte**: per ciascuna area portante del sistema, da quale problema si è partiti, quale fu la prima risposta, quali limiti ha rivelato e in quale soluzione si è infine consolidata. È nel percorso, più che nel punto d'arrivo, che si legge la logica del progetto.

Ogni decisione ha seguito lo stesso schema ricorrente, che vale la pena isolare perché è il vero metodo del sistema:

Diagramma 10 - Lo schema ricorrente con cui ogni decisione si è evoluta

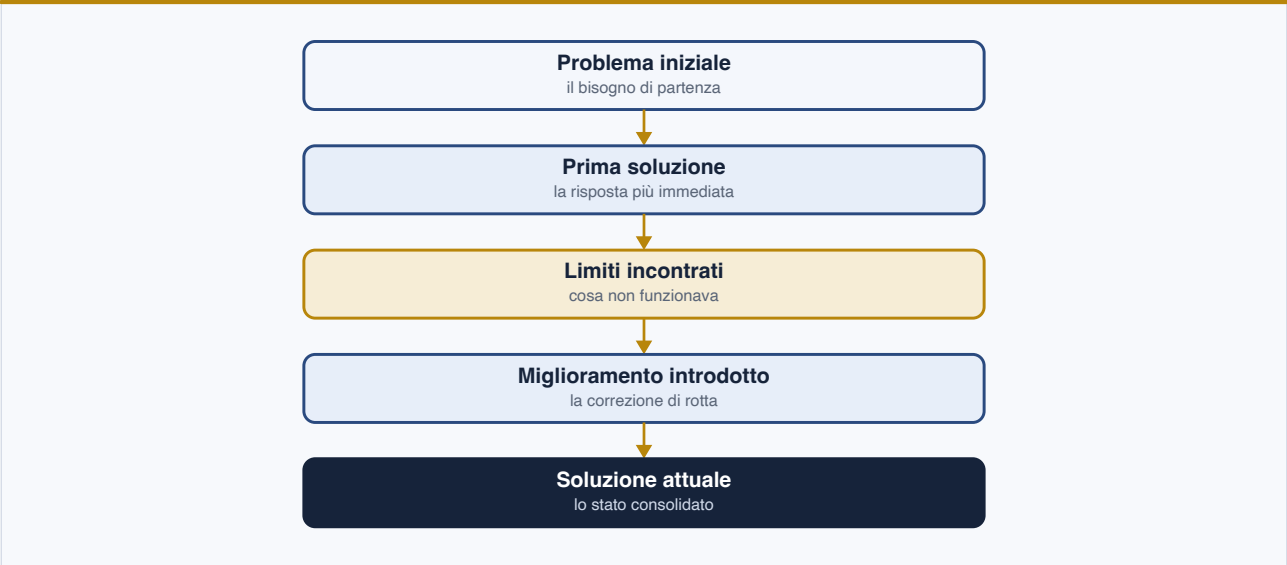


Diagramma 10 - Dal problema iniziale alla soluzione consolidata: ogni componente del sistema ha attraversato questo ciclo, spesso più di una volta.

Le otto evoluzioni portanti

Area	Problema -> prima soluzione	Limiti incontrati	Soluzione attuale
Modelli AI	Servire tutto con Claude (piano Max attivo)	Alla scadenza del Max il costo diventa insostenibile come default	DeepSeek operativo; Claude solo su richiesta esplicita
Routing	Un unico modello per ogni richiesta	Nessuna distinzione tra banale, sensibile e critico; spesa cieca	Router a plugin (before_dispatch) che sceglie il motore per tipo di task
Memoria	Ricaricare la cronologia grezza a ogni turno	Token gonfiati e contesto disordinato	Ricordi curati su file, distinti dalle note grezze, ispezionabili
Automazioni	Cron che si appoggiano a un modello AI	Costo e non determinismo per compiti che non lo richiedono	Report, backup e pulizie su launchd in Python, a costo AI nullo
Portfolio Intelligence	Prezzi sintetici e stime teoriche	Dati inaffidabili: NAV ancorato male, fallback silenziosi	Pricing reale verificato + motore deterministico, AI solo sul commento
Controllo costi	Spesa AI osservata a posteriori	Prezzi in configurazione gonfiati (~10x) falsavano i log	Tracker sui transcript + saldo provider; tariffe corrette e aggiornate
Integrazioni	Note operative dei dispositivi sempre in contesto	Contesto appesantito a ogni sessione	Indice leggero, schede dispositivo caricate solo quando servono

Area	Problema -> prima soluzione	Limiti incontrati	Soluzione attuale
Backup	Backup fragile, esiti non controllati	Fino a centinaia di fallimenti silenziosi non rilevati	Backup isolato e verificato, esiti monitorati end-to-end

Tabella 18.1 - Le otto aree portanti lette come evoluzione: da dove sono partite e dove si sono assestate. [ricostruzione]

Tre lezioni che hanno guidato le correzioni

ERRORE CORRETTO

I prezzi DeepSeek cablati in configurazione erano gonfiati fino a ~10x: i log di costo erano *plausibili ma falsi*. Corretti ai valori ufficiali verificati. Un numero credibile e sbagliato è più pericoloso di un errore evidente - da qui la caccia attiva ai guasti silenziosi.

LEZIONE APPRESA

Il costo di un agente con memoria non è dominato da ciò che scrive, ma da ciò che **rilegge** a ogni turno (~95% dei token in cache read). Capiro ha riorientato tutte le contromisure: contesto snello, memoria curata e modelli economici sul volume ripetuto.

DECISIONE PROGETTUALE

Il passaggio da Claude a DeepSeek come modello operativo non è stato un ripiego ma una scelta architettuale: il cambio di billing ha reso evidente che il default deve essere il modello più economico capace del compito, riservando il premium ai casi che lo giustificano, sempre su decisione esplicita.

Il filo comune di queste evoluzioni è che nessuna correzione è nata da una preferenza estetica: ognuna ha risposto a un limite **misurato** - un costo, un dato inaffidabile, un contesto troppo pesante - e ha lasciato il sistema più economico, più verificabile o più robusto di prima. È lo stesso metodo che il capitolo seguente formalizza in principi.

20. Principi architetturali e scelte progettuali

I capitoli precedenti descrivono *come* funziona il sistema. Questo capitolo spiega *perché* è stato progettato così: per ogni scelta, la motivazione e il vantaggio che produce. È nelle motivazioni - più che nei dettagli implementativi - che un'architettura dimostra di essere solida e replicabile, perché un principio si trasferisce a contesti diversi mentre un dettaglio resta legato al caso specifico.

Il principio che governa tutti gli altri è la **separazione netta tra ciò che deve essere deterministico e ciò che è bene affidare all'AI**: i numeri e le azioni ripetibili sono codice; l'AI interpreta, sintetizza e scrive. La figura seguente mostra questa divisione del lavoro, che ritorna in ogni modulo del sistema.

Diagramma 9 - Divisione del lavoro: deterministico (codice) e AI (modelli)

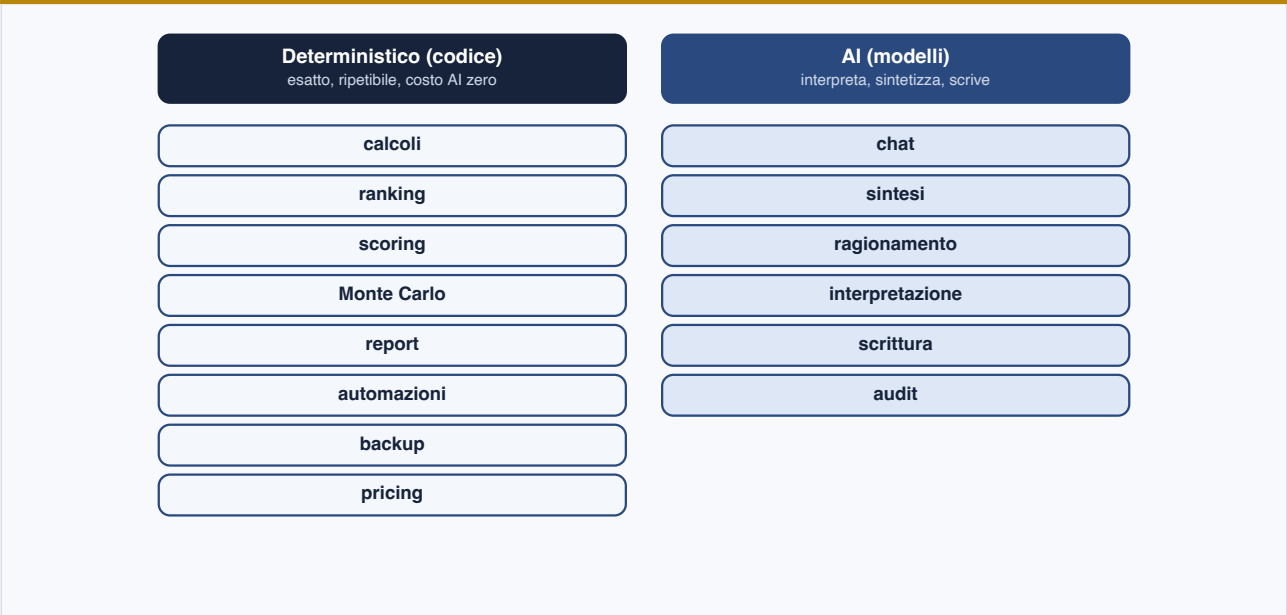


Diagramma 9 - Ciò che deve essere esatto e ripetibile resta codice (zero token); l'AI si occupa di linguaggio, ragionamento e interpretazione. È la linea di confine che regge l'intera architettura.

Le decisioni progettuali a confronto

Il modo più utile di leggere un'architettura non è l'elenco di ciò che è stato adottato, ma il confronto con le alternative scartate: per ogni scelta strutturale, quale opzione è stata messa da parte, per quale ragione, e quale beneficio concreto ha prodotto la soluzione adottata. La tabella raccoglie le decisioni portanti del sistema in questa forma. Ogni riga è una decisione difendibile, non una preferenza: è qui, più che nei dettagli implementativi, che un'architettura dimostra di essere solida e replicabile, perché un principio si trasferisce a contesti diversi mentre un dettaglio resta legato al caso specifico.

Decisione	Alternativa scartata	Motivo dello scarto	Soluzione adottata	Beneficio ottenuto
Base sull'agente OpenClaw	Stack proprietario o framework legato a un singolo modello	Lock-in di fornitore e di modello; difficile cambiare componente senza riscrivere	Telaio aperto con canali, plugin, memoria e routing intercambiabili	Indipendenza dal fornitore e libertà di evolvere ogni parte
Esecuzione in locale dove conta	Tutto in cloud, su API di terzi	Dati sensibili esposti, costo per ogni chiamata, dipendenza dalle politiche altrui	Modelli e memoria locali per sensibile e routine; cloud solo dove conviene	Privacy, costo zero sul locale e meno dipendenza esterna
DeepSeek come modello di default	Claude (o altro premium) come modello predefinito	Costo per token molto più alto su un volume di richieste in larga parte ordinarie	Default economico; il premium si attiva solo su richiesta esplicita	Spesa AI ridotta a parità di lavoro svolto, senza perdere qualità dove serve

Decisione	Alternativa scartata	Motivo dello scarto	Soluzione adottata	Beneficio ottenuto
Telegram come interfaccia unica	Interfaccia web o app dedicata da sviluppare	Sviluppo e manutenzione di una UI, maggiore superficie da proteggere	Un solo canale già pronto, con vocali, file e notifiche native	Zero sviluppo UI, ingresso unico e meno superficie d'attacco
Logica deterministica separata dall'AI	Delegare anche calcoli e report al modello	Numeri non riproducibili, errori silenziosi, token spesi per compiti esatti	Conti, report e backup in codice; l'AI solo per linguaggio e interpretazione	Risultati esatti e ripetibili, con risparmio di token
Routing esplicito dei modelli	Un unico modello per ogni tipo di richiesta	O si paga troppo per compiti banali, o si usa un modello debole per quelli complessi	Regole che assegnano il modello giusto a ogni categoria di richiesta	Costo sotto controllo e qualità adeguata caso per caso
Automazioni schedate con launchd	Loop AI periodici o scheduler di terze parti	Costo AI ricorrente e dipendenza da servizi esterni per compiti fissi	Job di sistema nativi che eseguono codice deterministico a orari fissi	Routine robusta a costo AI nullo, senza dipendenze aggiuntive
Routing via hook before_dispatch	Modificare il core dell'agente o mantenerne un fork	Manutenzione fragile e aggiornamenti difficili a ogni nuova versione	Plugin che intercetta la richiesta prima dell'invio e sceglie il modello	Routing personalizzato senza toccare il core, facile da aggiornare
Claude attivabile solo su richiesta	Modello premium disponibile come fallback automatico	Rischio di spesa premium silenziosa e non voluta	Premium escluso dalla scala automatica; si attiva solo con comando esplicito	Nessun costo premium involontario, controllo totale sulla spesa
Memoria come file leggibili	Database o servizio esterno di memoria	Scatola nera, dipendenza esterna, contenuto poco ispezionabile	Note grezze e ricordi curati in file di testo, distinti e versionabili	Memoria ispezionabile, correggibile a mano e portabile
Backup cifrato in locale	Backup su cloud di terze parti	Dati personali e di sistema affidati a un fornitore esterno	Archivio cifrato del workspace schedato con launchd	Continuità del dato senza esporlo a terzi
Portfolio Intelligence deterministico	Calcoli finanziari affidati direttamente al modello	Numeri non riproducibili e rischio di cifre inventate su dati reali	Metriche calcolate in Python con test; l'AI solo per il commento	Dati esatti e verificabili, con consumo di token ridotto

Tabella 19.1 - Le decisioni strutturali del sistema messe a confronto con le alternative scartate. Ogni scelta è ricondotta al motivo dello scarto e al beneficio concreto ottenuto. [ricostruzione]

Principio di redazione del documento

DECISIONE PROGETTUALE

In caso di conflitto tra accuratezza tecnica, completezza storica ed eleganza grafica, privilegiare sempre **accuratezza tecnica e verificabilità documentale**. Questo documento è marcato per livello di certezza - [verificato], [ricostruzione], [stima] - e quando un dato non è verificabile con sicurezza viene dichiarato come tale anziché stimato. La credibilità e la durata nel tempo del white paper contano più della sua forma.

Appendice

A. Glossario

Termine	Significato
OpenClaw	Piattaforma che fa girare l'agente: gateway, plugin, canali, modelli
Agente	Sistema che percepisce, decide e compie azioni reali (non solo testo)
Routing	Meccanismo che sceglie quale modello usare per ogni richiesta
Modalità gratis	Stato in cui l'agente risponde col modello locale a costo zero
Deterministico	Calcolo con risultato esatto e ripetibile, senza AI
launchd	Gestore di servizi/automazioni nativo di macOS
Ollama	Runtime per modelli AI locali sul Mac
LLM	Large Language Model: modello linguistico di grandi dimensioni
STT / TTS	Speech-to-Text (voce->testo) / Text-to-Speech (testo->voce)
Fallback	Modello di riserva usato se il principale non risponde
NAV	Net Asset Value: valore di un fondo/portafoglio
Monte Carlo	Simulazione probabilistica di scenari futuri

B. Cartelle e file principali

- **~/openclaw/** - radice della piattaforma (config, sessioni, plugin); **~/openclaw/workspace/** - casa dell'agente: identità, memoria, script.
- **workspace/memory/** - note giornalieri e changelog; **workspace/device-notes/** - schede dispositivi; **workspace/plugins/gratis-local/** - plugin di routing locale.
- **openclaw.json** - configurazione (modelli, canali, plugin); **MEMORY.md / SOUL.md / AGENTS.md / USER.md** - regole core, persona, operatività, profilo.
- **portfolio_*.py** - moduli Portfolio Intelligence; **spend_report_send.py** - report spesa serale; **.gratis_mode / .gratis_model** - stato della modalità locale.

C. Servizi launchd principali

Servizio	Funzione	Cadenza
ai.openclaw.gateway	Gateway dell'agente	Sempre attivo (KeepAlive)
com.hetepi.portfolio-botd-unified	Bot Portfolio multi-utente	Sempre attivo (KeepAlive)
com.hetepi.spend-report	Report spesa AI su Telegram	Ogni giorno 23:00
com.hetepi.oc-backup	Backup cifrato del workspace	Settimanale (dom. 04:00)
com.hetepi.inbound-cleaner	Cancella i media in ingresso	Ogni 120 s
com.hetepi.mac-cleanup	Pulizia del Mac	Ogni giorno 03:00
com.hetepi.portfolio-monitor / -advice / -weekly	Alert e report di portafoglio	Schedulati
com.hetepi.openclaw-watchdog	Supervisione del gateway	Periodico

D. Comandi terminale principali

Comando / operazione	Scopo
openclaw gateway (via launchd)	Avviare il gateway (gestito da launchd)
launchctl load/unload ~/Library/LaunchAgents/.plist	Attivare/fermare un servizio
ollama run qwen3:8b-q4_K_M	Eseguire un modello locale
python3 spend_monitor.py --reale	Spesa AI reale del giorno
gratis on / gratis off (in chat)	Entrare/uscire dalla modalità locale a costo zero

E. Nota sui segreti

Per scelta, questo documento non contiene alcun valore segreto: token del bot, chiavi API dei provider, chiave di web search, passphrase di backup. Dove tali elementi sono citati, se ne descrive solo l'esistenza e la funzione. I segreti risiedono in file a permessi ristretti o nella configurazione, fuori dalla portata di questo testo.

F. Audit tecnico finale

Questa appendice è una checklist di verifica dei componenti del sistema. Per ciascuno indica lo stato operativo, il livello di verifica adottato, la fonte concreta da cui la verifica è stata tratta e la copertura raggiunta. È lo strumento che permette a un lettore esterno di distinguere ciò che è stato direttamente ispezionato da ciò che è ricostruito.

Componente	Stato	Livello di verifica	Fonte della verifica	Copertura
Modelli (DeepSeek, Claude, Ollama)	Attivo	[verificato]	openclaw.json + catalog provider	Completa
Routing (plugin gratis-local)	Attivo	[verificato]	plugins/gratis-local/index.js	Completa
Plugin / hook before_dispatch	Attivo	[verificato]	Codice del plugin	Completa
Memoria (note + curata)	Attivo	[verificato]	MEMORY.md + memory/ + file per tema	Completa
Servizi launchd e automazioni	Attivo	[verificato]	LaunchAgents + stato servizi + log	Completa
Portfolio Intelligence	Attivo	[verificato]	portfolio_*.py + suite test (170+ smoke)	Alta
Monitoraggio token e spesa	Attivo	[verificato]	spend_monitor.py + deepseek_balance_log.json	Completa
Tariffe modelli	Aggornate	[verificato]	RATES + aggiornamento da listini pubblici	Completa
Integrazioni (Google, TV, robot, luci)	Attivo	[verificato]	Schede device-notes + script	Alta
Backup cifrato	Attivo	[verificato]	Servizio oc-backup + script cifratura	Alta
Pulizia media inbound	Attivo	[verificato]	Servizio inbound-cleaner (120 s)	Completa
Tempi di risposta cloud	Da misurare	[stima]	Non cronometrati alla data	Parziale
Sequenza di prima installazione	Storico	[ricostruzione]	Stato finale coerente, comandi non registrati	Parziale

Appendice F - Audit tecnico finale: componente -> stato -> livello di verifica -> fonte -> copertura.

HETEPI AI · ECOSISTEMA OPENCLAW

Conclusioni

Hetepi AI è un assistente personale trasformato in **agente operativo**: un sistema che, a partire da un'unica interfaccia Telegram, percepisce le richieste, sceglie come rispondere, esegue azioni reali e ne rende conto. È stato costruito per superare i limiti di un chatbot generico - un solo modello costoso, nessuna memoria, nessuna azione - e per farlo entro vincoli espliciti di costo e di privacy fissati fin dall'inizio.

L'impianto architetturale poggia su pochi principi coerenti. **OpenClaw** fa da orchestratore: riceve i messaggi, instrada verso il modello adeguato tramite un routing esplicito e tiene insieme memoria, tool e automazioni. La **separazione tra AI e logica deterministica** è la scelta portante: i calcoli ripetibili, i conteggi, i report e i backup vivono nel codice Python e non consumano modelli, mentre l'AI è riservata a ciò che richiede interpretazione e sintesi.

Da questa separazione discende il **controllo dei costi**: il default è il modello più economico in grado di svolgere il compito (DeepSeek Flash), i modelli più cari si attivano solo su richiesta esplicita, e ciò che è banale o sensibile resta in locale tramite Ollama, a costo zero. La **memoria persistente** - note giornaliere e memoria curata per tema - dà continuità e personalizzazione tra sessioni, mentre le **integrazioni** con Google Workspace, dispositivi di casa e modulo finanziario rendono concreto il salto da conversazione ad azione.

Il modello è **replicabile**: il telaio - gateway, routing, memoria, fallback, automazioni deterministiche - resta invariato, e ad adattarsi sono solo i modelli configurati e le automazioni specifiche. Questo lo rende applicabile a contesti professionali e aziendali in cui contano il controllo dei dati, la riduzione della dipendenza da fornitori esterni e la prevedibilità della spesa. Allo stato attuale il sistema è **maturo e in produzione**: agente, automazioni, backup cifrato e modulo verticale Portfolio Intelligence sono operativi, con un debito tecnico noto, circoscritto e tracciato. Questo documento ne fissa l'architettura e il ragionamento, perché resti leggibile e riproducibile nel tempo.

Andrea Sforna

Progetto e documento Hetepi AI aggiornati al 19 giugno 2026.